



WS51F60/70 系列 MCU

应用指南

此文档主要描述如何使用 WS51F60/70 系列 51 内核的单片
机。本文档第三章描述电路设计，主要介绍常用本 MCU 常用的电路，
特点是触摸按键电路，LED/LCD 电路。第四章描述程序编写，给出每
一个外设的使用说明。

版本：1.3

深圳前海维晟智能技术有限公司

电话：+086-0755-23310575

<http://www.wisesun.com>

目录

目录.....	2
1 WS51F60/70 系列芯片电气参数注意项.....	5
2 WS51F60/70 系列芯片烧写注意项.....	5
3 电路设计注意事项.....	5
3.1 复位电路.....	5
3.2 晶振电路.....	5
3.3 下载/仿真电路.....	6
3.4 触摸按键电路.....	6
3.5 LED 电路.....	7
3.6 LED 与触摸按键共用引脚电路.....	8
3.7 LCD 电路.....	9
3.8 I2C 电路.....	9
4 软件编写的注意事项.....	10
4.1.1 时钟系统.....	10
4.1.2 时钟系统介绍.....	10
4.1.3 例程代码.....	10
4.2 复位系统.....	11
4.2.1 复位系统.....	11
4.2.2 例程代码.....	11
4.3 外部中断.....	12
4.3.1 外部中断描述.....	13
4.3.2 例程代码.....	13
4.4 低功耗程序设计.....	16
4.4.1 IDLE 介绍.....	16
4.4.2 STOP 介绍.....	16
4.4.3 例程代码.....	17
4.5 Timer0/1.....	20
4.5.1 Timer0/1 介绍.....	20
4.5.2 例程代码.....	20
4.6 Timer2.....	21
4.6.1 Timer2 介绍.....	21
4.6.2 例程代码.....	22
4.7 UART0.....	25
4.7.1 UART0 的 4 种工作模式波特率公式.....	25
4.7.2 定时器波特率分频设置(修改分频系数).....	26
4.7.3 参考代码.....	26
4.8 PWM.....	28
4.8.1 PWM 工作原理.....	29
4.8.2 例程代码.....	29
4.9 I2C.....	31
4.9.1 I2C 介绍.....	31
4.9.2 I2C 通信原理.....	31

4.9.3 I2C 工作模式.....	31
4.9.4 例程代码作为主机.....	32
4.9.5 例程代码作为从机.....	38
4.10 WDT.....	41
4.10.1 WDT 介绍.....	41
4.10.2 例程代码.....	41
4.11 WKT.....	42
4.11.1 WKT 介绍.....	42
4.11.2 例程代码.....	42
4.12 DAC.....	43
4.12.1 DAC 介绍.....	43
4.12.2 功能描述.....	43
4.12.3 例程代码.....	43
4.13 LVD.....	43
4.13.1 简介.....	43
4.13.2 功能描述.....	44
4.13.3 例程代码.....	44
4.14 软件 LED 驱动.....	44
4.14.1 软件 LED 介绍.....	44
4.14.2 LED 显示原理.....	44
4.14.3 LED 与 Touch Key 共用方法.....	45
4.14.4 轮询 LED 例程代码.....	45
4.14.5 共阳数码管例程代码.....	52
4.15 软件 LCD 驱动.....	53
4.15.1 软件 LCD 描述.....	53
4.15.2 例程代码.....	53
4.16 Flash 数据区.....	59
4.16.1 Flash 区介绍.....	59
4.16.2 Code 区擦除函数.....	60
4.16.3 Code 区写数据函数.....	60
4.16.4 Code 区读数据函数.....	61
4.16.5 Data 区擦除函数.....	62
4.16.6 Data 区写数据函数.....	62
4.16.7 Data 区读数据函数.....	63
4.16.8 查看当前 CODE 扇区累加和函数.....	64
4.16.9 查看当前 DATA 扇区累加和函数.....	64
4.17 NVR 数据区.....	64
4.17.1 NVR 数据区简介.....	64
4.17.2 NVR 例程代码.....	64
4.18 EEPROM 数据区.....	66
4.18.1 EEPROM 数据区介绍.....	66
4.18.2 例程代码.....	66
4.19 触摸按键介绍.....	68
4.19.1 触摸按键使用.....	68

4.20 其他.....	69
4.20.1 程序启动空间测试.....	69
5 芯片应用设计注意.....	70

1 WS51F60/70 系列芯片电气参数注意事项

参数	最小值	最大值	单位
直流供电电压	-0.3	6	V
I/O 端口输入电压	-0.3	VDD+0.3	V
工作环境温度	-40	105	°C
储存温度	-45	125	°C
CPU 工作频率	-	16	MHz
最大通过电流	-	300	mA

2 WS51F60/70 系列芯片烧写注意事项

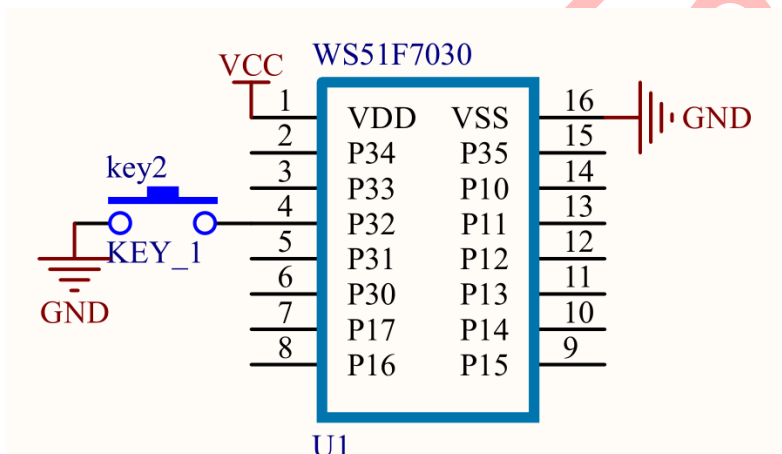
- 带电烧写的时候负载电流不能超过，一定要带电烧录，使用复位引脚

3 电路设计注意事项

3.1 复位电路

此单片机的复位电路如下图所示，芯片有内部上拉，可以省掉上拉电阻，在要求低的条件下也可以省略电容。

在使用了所有引脚的情况下，可以先把 P32 拉低再上电，这样子只要程序有 1MS 的延时就能实现下载程序。



3.2 晶振电路

此单片机集成精度±1%的内部 16MHz RC 振荡器(HRC)和±1%的内部 32K RC 振荡器(LRC)。

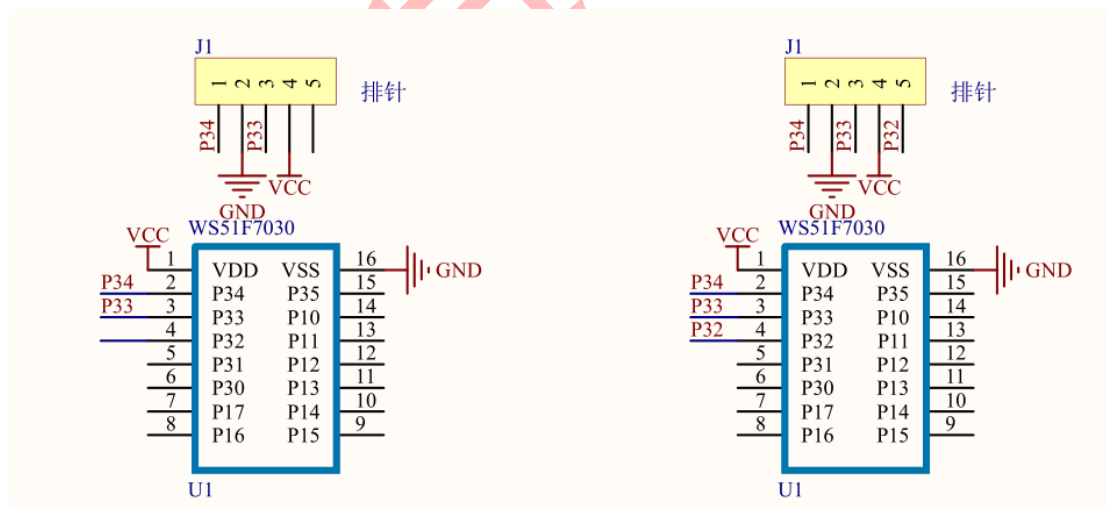
在使用相应的模块会自动开启对应的时钟。

3.3 下载/仿真电路

此单片机使用 I2C 协议进行下载/仿真，建议不要关闭与 I2C 相关的功能。如 I2C 中断 (IE 中的 EINT2)，I2C 相关寄存器，I2C 引脚 P33F, P34F。如下左图所示

如果程序中将下载相关寄存器关闭。如果还要下载程序则需要满足以下条件（1）需要在 main() 函数里加一个 50MS 以上的空操作；（2）将外部能供电的设备拔掉，电源由下载器供电或者使用外部供电(2.8V 到 5.0V 之间)，P32 设置为复位引脚，下载器 RST 线连接到芯片 RST。

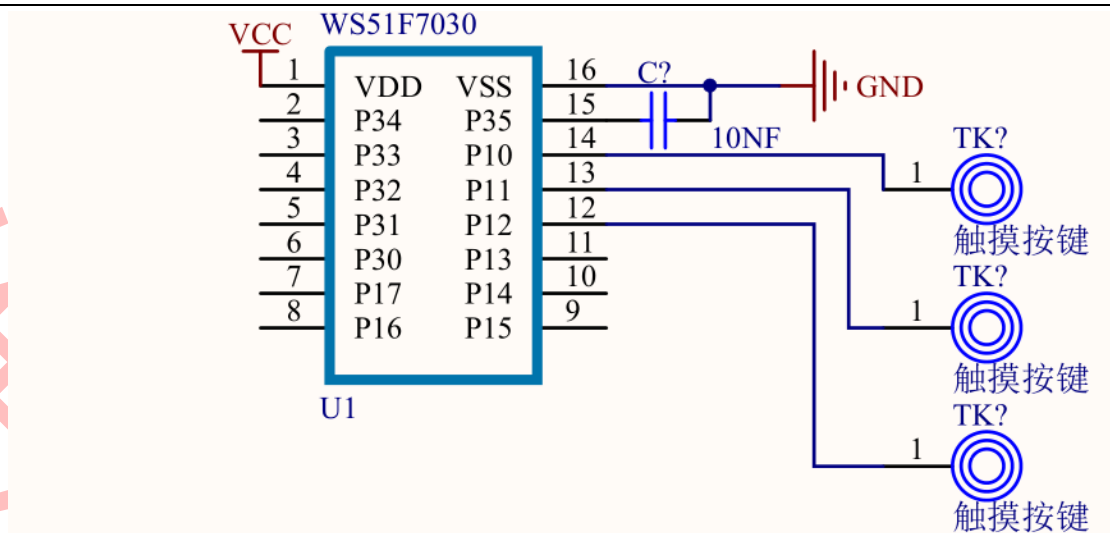
```
Void main(void)
{
    HRC|=0X80;
    LRC|=0X80;
    SCCON=0;
    Delay50Ms();
    //用户程序
    ****
}
```



3.4 触摸按键电路

此单片机需要在 P35 与 GND 之间连接一个 10NF 到 47NF 的电容，其他引脚直接接触摸按键的焊盘即可。

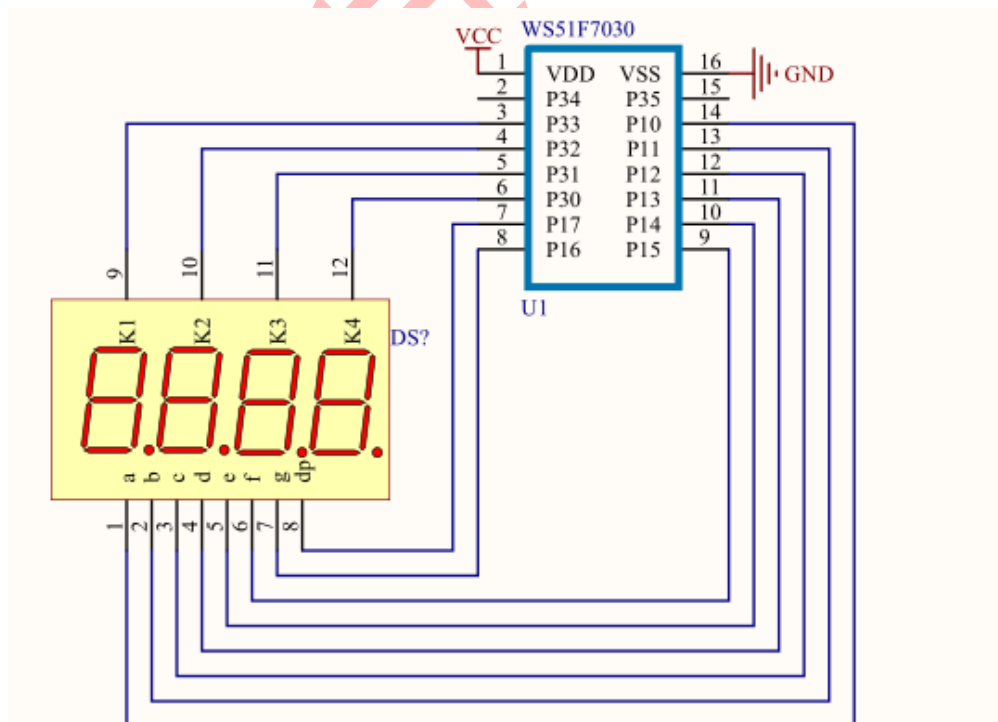
触摸按键的工作频率较高，因此请做好不同通道之间的隔离处理。



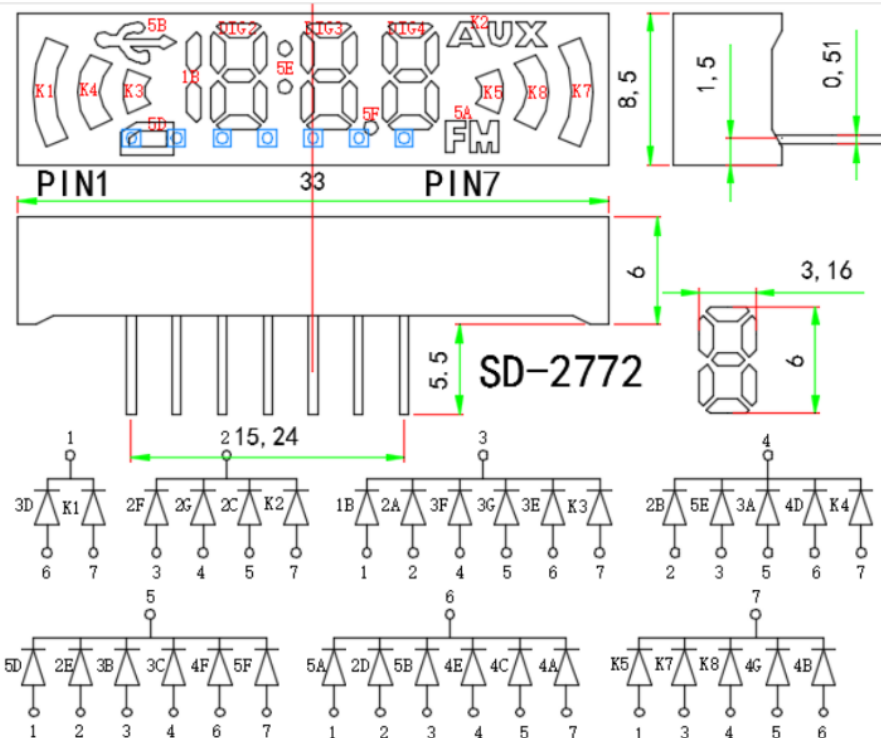
3.5 LED 电路

普通数码管(3641XS)是共阴数码管还是共阳数码管都可以使用一样的电路连接，只需要在程序上修改对应的寄存器即可。MCU 与数码管引脚可以任意使用 P10-P17，P30-P33 这 12 个引脚，修改引脚需要修改的是对应的显示数组。

此处给出一个连接的示意图如果是使用请根据



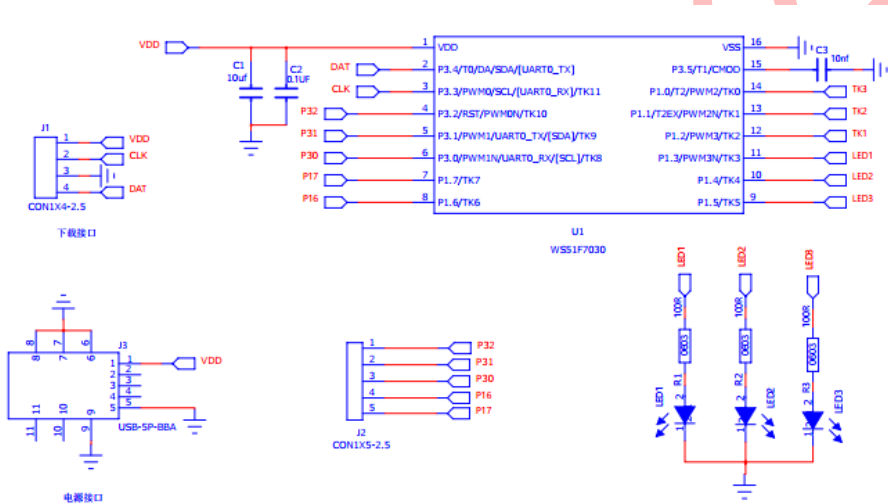
特殊数码管其内部设计与普通数码管不同，因此只需要使用少量的引脚即可实现与数码管一样的显示功能，电路连接如下图所示。



3.6 LED 与触摸按键共用引脚电路

注意：LCD 与触摸按键引脚不能共用引脚，另外，LED 屏与触摸按键引脚不能共用引脚，只能与单独灯引脚共同使用。

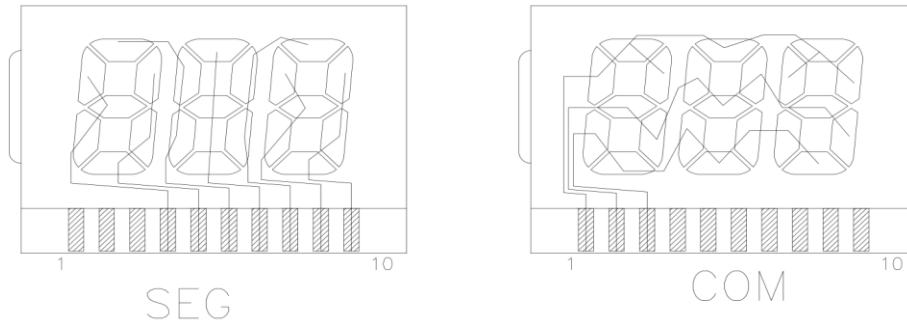
此 MCU 的触摸按键可以与 LED 灯共用引脚，使用分时复用的方法进行 LED 显示与触摸按键检测，触摸按键与 LED 使用一样的引脚因此会导致触摸按键的引脚被拉长，因此可能会影响触摸按键的效果。LED 引脚必须接到 IO 口，不能直接接到 VCC 或者 GND。



3.7 LCD 电路

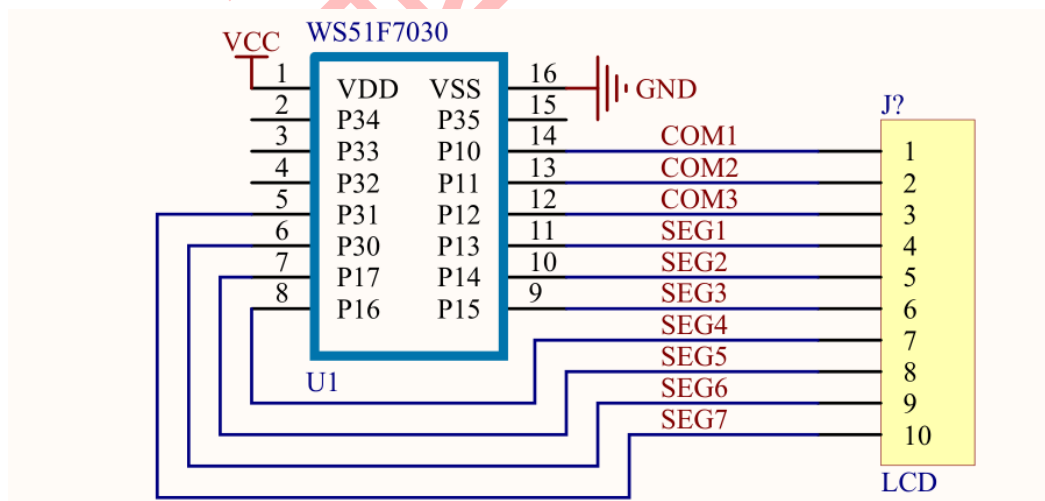
注意：LCD 与触摸按键引脚不能共用引脚。

LED 氛围 SEG 引脚和 COM 引脚，具体分布图请参照厂家的数据手册。此处给出一个示意图。



此 MCU 仅能点亮 1/2BIAS 的 LCD 屏幕，在选购的时候请不要选购 1/3BIAS 或者 1/4BIAS 的 LCD 屏幕。

此 MCU 驱动 LCD 的 COM 脚的引脚为 P10-P13，SEG 引脚为普通 IO 口，具体引脚可以参考数据手册里的“IO 端口复用功能映射表”。

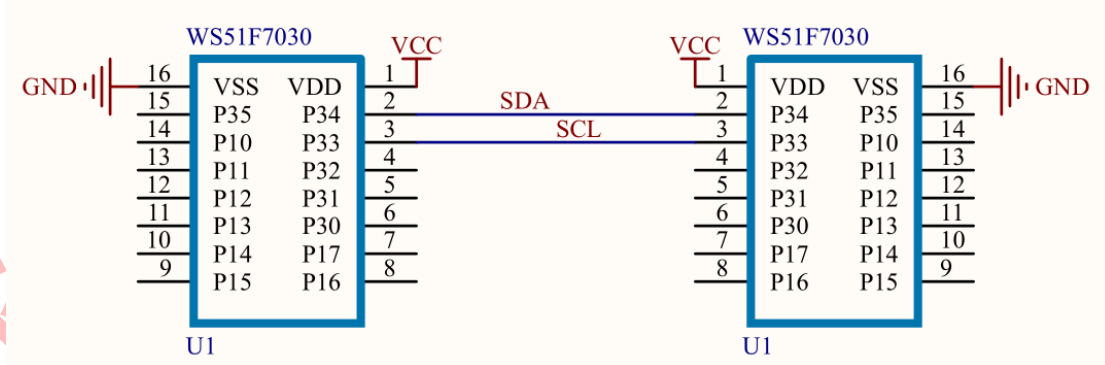


3.8 I2C 电路

注意：如果程序占用了 I2C 接口，则芯片不能进行仿真功能。

此 MCU 可以设置内部上拉，如 I2C 频率不是很快的情况(如 100K)下可以选择内部上拉电阻。IIC 频率较快的时候请使用外部上拉。此 MCU 支持主机或者从机模式，主机模式或从机模式可以软件配置进行切换。

需要注意的是下载器使用的是 I2C 的 P33 和 P34 脚烧录程序的。如果程序配置了 P33/P34 非 IIC 功能，或关掉了 I2C 相关寄存器，需要在程序里面一上电加 50ms 的延时防止芯片锁死，不能重复烧写代码。



4 软件编写的注意事项

本 MCU 外设的一般使用方法为配置引脚，设置相关寄存器，设置中断，编写中断处理函数这四部分。

MCU 内外设可直接配置相关寄存器，详细芯片规格书资料 WS51F60/70 系列。

4.1.1 时钟系统

4.1.2 时钟系统介绍

WS51F60/70 系列 系列芯片共支持以下时钟源：

内部 16MHz RC 振荡器 HRC

内部 32KHz RC 振荡器 LRC

此 MCU 的时钟在使用对应的功能时会自动开启，开启时钟后如果关闭了外设，时钟仍然继续工作。具体的外设与时钟对应关系请参考数据手册的” 外设时钟映射表”。如使用了使用 WKT 则自动开启 LRC。若系统仅开启 WKT 然后关闭 WKT，则关闭 WKT 后 LRC 仍然工作。关闭时钟的操作请参考本文档的“低功耗程序设计”。

4.1.3 例程代码

```

/***** 系统时钟配置 HRC *****/
SCCON = 0x00; //选择高速时钟 HRC, 不分频
HRCON |= 0x80; //使能内部 HRC

/***** 系统时钟配置 LRC *****/
SCCON = 0x80; //选择高速时钟 LRC, 不分频
LRCON |= 0x80; //使能内部 LRC
    
```

4.2 复位系统

4.2.1 复位系统

芯片内部有 5 种复位源：上电复位（POR）、掉电复位（BOR）、看门狗复位（WDT）、低电压检测复位（LVD）、外部硬复位和内部软复位。

/******

注意事项：

- 1、低电压检查(LVD), 复位标志(LVDF),清除 LVDFCON 中相关标志自动清除
- 2、WDT 复位标志(WDRF),喂狗刷新 WDT 自动清除
- 3、其他复位标志写 0 清
- 4、如果产生了 LVD 复位,不会复位寄存器 LVDFCON(LVDF=1),只要不手动清标志位 LVDF, 芯片会一直处于复位状态, 当供电超过低压检测值后, 上电 LVDF 会自动清

*****/

4.2.2 例程代码

```
#define RST_SWRF 0x01 //软复位标志,写 0 清
#define RST_HWRF 0x02 //硬复位标志,写 0 清
#define RST_PDRF 0x04 //上电或掉电复位标志,写 0 清
#define RST_WDRF 0x08 //WDT 复位标志,喂狗刷新 WDT 自动清除
#define RST_LVDF 0x10 //LVD 复位标志,清除 LVDFCON 中相关标志自动清除
```

/******

*函数名称: void RstSys_Test(void)

*函数功能: 查看复位标志

*入口参数: void

*出口参数: void

*****/

```
void RstSys_Test(void)
```

```
{
```

```
    unsigned char i=0;
```

```
    /****** UART0 模式 1（波特率：9600）*****
```

```
    P31F = 6;//TX----
```

```
    InitUart0_Timer2(UART0_BAUDRATE_9600_T2); //Uart0 波特率 9600
```

```
    // 此处可以读 RSTFLG, 来确认是何种复位
```

```
    printf("RSTFLG = 0X%bx\r\n",RSTFLG); //查看复位标志
```

```
    RSTFLG=0; //清标志
```

```
    /*LVD*/
```

```
    /*WDT 复位开启*/
```

```
WDTCON=0X07;//enable WDT reset 1s
```

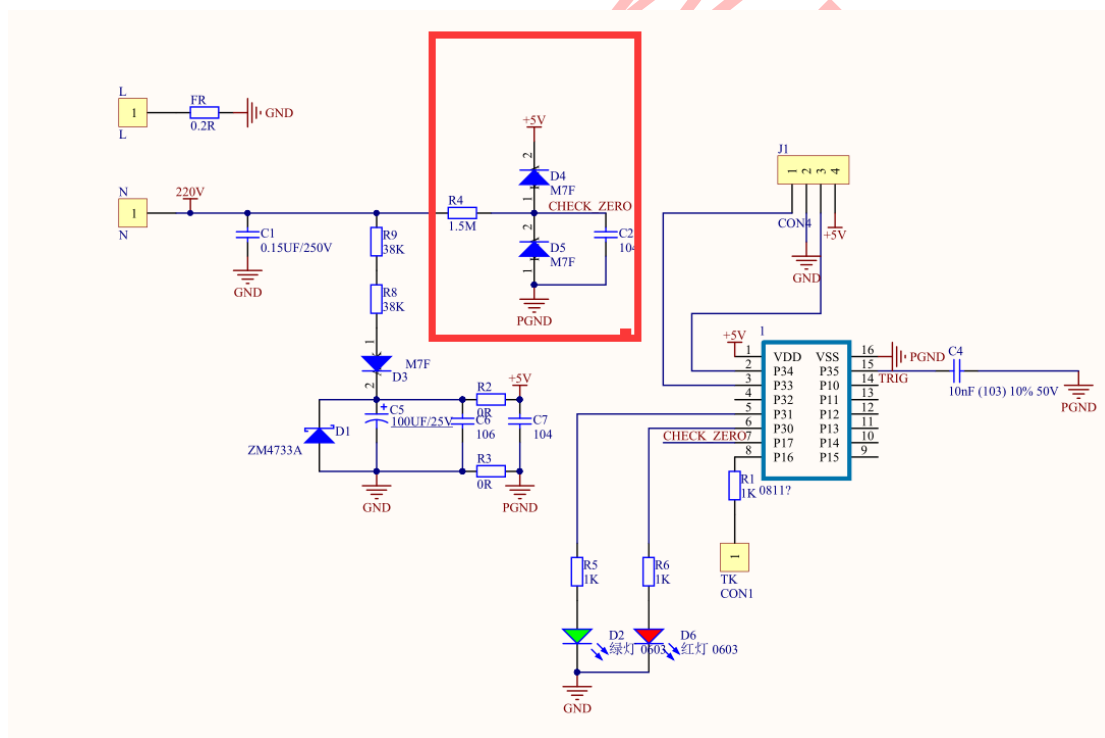
```
WDTFLG=0XA5;//open wdt
```

```
P10F = 2;
P30F = 1;
while(1)
{
    P10 = ~P10;

    if(P30 == 0)
    {
        delay(10);
        if(P30 == 0)
        {
            while(P30==0);
            PCON = 0x20; //软件复位
        }
    }
}
```

4.3 外部中断

如果是使用 IO 口检测市电，请在 IO 口上接上二极管。如图所设计：



4.3.1 外部中断描述

除了标准 8051 的 INT0 和 INT1 以外，系统还扩展了 2 个中断入口 INT2/INT3 作为外部中断 EP0/EP1。这四个中断可以配置为任意 IO 引脚。为了提高外部中断的抗干扰性能，每个外部中断都集成了模拟滤波器。

注意：8051 的 INT0 和 INT1 支持电平触发和下降沿触发中断。INT2/INT3 可以选择为上升沿、下降沿、双沿触发中断。因此 INT2/INT3 不支持电平触发，INT0 和 INT1 不支持上升沿触发。这四个中断一次最多能配置为四个引脚的中断。

EPIF 为 EP0/EP1 外部中断状态寄存器，EP0/EP1 对应的各个配置寄存器为 EP0CON/EP1CON。

配置 INT0 的过程：配置 TCON[0: 1] 选择触发模式和使能，IT0CON 选择引脚和触发方式，使能 IE，编写中断处理函数。中断标志自动清除。

配置 INT1 的过程：配置 TCON[2: 3] 选择触发模式和使能，IT1CON 选择引脚和触发方式，使能 IE，编写中断处理函数。中断标志自动清除。

配置 INT2 的过程：配置 EP0CON 选择引脚和触发方式，使能 IE 和 EPIE，编写中断处理函数。中断标志不会自动清除，标志位在 EPIF 寄存器里。

配置 INT3 的过程：配置 EP1CON 选择引脚和触发方式，使能 IE 和 EPIE，编写中断处理函数。中断标志不会自动清除，标志位在 EPIF 寄存器里。

4.3.2 例程代码

INT0-code 如下:

```
#define IT_PIN_P10  0X08
#define IT_PIN_P11  0X09
#define IT_PIN_P12  0X0A
#define IT_PIN_P13  0X0b
#define IT_PIN_P14  0X0c
#define IT_PIN_P15  0X0d
#define IT_PIN_P16  0X0e
#define IT_PIN_P17  0X0f
#define IT_PIN_P30  0X18
#define IT_PIN_P31  0X19
#define IT_PIN_P32  0X1A
#define IT_PIN_P33  0X1b
#define IT_PIN_P34  0X1c
#define IT_PIN_P35  0X1d

/*****
*函数名称: void Int0SetMode0(void)
*函数功能: INT0 中断模式 配置函数
*入口参数:
*出口参数:
*****/

void Int0SetMode0(void)
```

```

{
    /***** INT0 中断模式 0 *****/
    IT0CON=0X00|INT_PIN;    //00 Pin---下降沿  --低电平
    // IT0CON=0X40|INT_PIN;    //10 Pin---上沿    --高电平
    // IT0CON=0X80|INT_PIN;    //01 Pin---双沿    --保留（低电平）
    // IT0CON=0XC0|INT_PIN;    //01 Pin---双沿        --保留（低电平）

    TCON=3;//使能外部中断 0 在输入管脚下降沿时触发
    // TCON=2;//使能外部中断 0 在输入管脚低电平时触发
}

```

*函数名称: void Int1SetMode0(void)

*函数功能: INT1 中断模式 配置函数

*入口参数:

*出口参数:

*****/

```
void Int1SetMode0(void)
```

```
{
    /****** INT1 中断模式 0 *****/
    // IT1CON=0X00|INT_PIN;    //00 Pin---下降沿---
    IT1CON=0X40|INT_PIN;    //10 Pin---上沿 ---
    // IT1CON=0X80|INT_PIN;    //01 Pin---保留（低电平）---双沿
    // IT1CON=0XC0|INT_PIN;    //01 Pin---保留（低电平）---双沿
    TCON=0x0c;//外部中断 1 在输入管脚下降沿时触发
    // TCON=0x08;//外部中断 1 在输入管脚低电平时触发
}
```

/*****

*函数名称: void Int2SetMode0(void)

*函数功能: INT2 中断模式 配置函数

*入口参数:

*出口参数:

*****/

```
void Int2SetMode0(void)
```

```
{
    /****** INT2(EP0) 中断模式 0 *****/
    EP0CON=0X00|INT_PIN;    //00 Pin---下降沿---
// EP0CON=0X40|INT_PIN;    //10 Pin---上沿   ---
// EP0CON=0X80|INT_PIN;    //01 Pin---双沿---
// EP0CON=0XC0|INT_PIN;    //01 Pin---双沿---
}
```

*函数名称: void Int3SetMode0(void)

*函数功能: INT3 中断模式 配置函数

*入口参数:

*出口参数:

*****/

void Int3SetMode0(void)

```
{
    /****** INT3(EP1) 中断模式 0 *****/
    // EP1CON=0X00|INT_PIN;    //00 Pin---下降沿---
    // EP1CON=0X40|INT_PIN;    //10 Pin---上沿 ---
    // EP1CON=0X80|INT_PIN;    //01 Pin---双沿---
    EP1CON=0XC0|INT_PIN;    //01 Pin---双沿---
}

/****** EP0~EP3 中断使能设置 *****/
// EP1IE = 1;    /* EP1 中断使能信号 */
// EP0IE = 1;    /* EP0 中断使能信号 */

EINT3 = 1;    /* EP1 用于 Touch Key/WDT/LVD/WKT/EP1 中断 */
EINT2 = 1;    /* EP0 用于 IIC/Timer2/PWM/EP0 中断 */
EINT1 = 1;    /* INT1 中断 1 使能控制位（中断 1 用于外部中断 1） */
EINT0 = 1;    /* INT0 使能控制位（中断 0 用于外部中断 0） */
IE|=0x80;//总中断使能
```

中断服务函数如下:

*****中断函数*****/

void IT1_isr() interrupt 2

```
{
}
```

void IT0_isr() interrupt 0

```
{
}
```

void EP1_isr() interrupt 6

```
{
    if((EPIF&0x02) == 0x02)
    {
        EPIF &= 0xFD;
    }
}
```

void EP0_isr() interrupt 5

```
{
    if((EPIF&0x01) == 0x01)
    {
```

```
EPIF &= 0xFE;
```

```
}
```

```
}
```

4.4 低功耗程序设计

此 MCU 低功耗设计有 IDLE 模式和 STOP 模式。IDLE 模式是一种相对来说功耗大一点的睡眠模式。STOP 模式在有的芯片上也叫做 SLEEP 模式，此模式是真正的睡眠模式，能将 MCU 的功耗将至最低(低于 3UA)。

注意：用户可以自己设置睡眠之后的 IO 的状态。

4.4.1 IDLE 介绍

在 IDLE 模式下，CPU 将停止工作。进入 IDLE 模式前，除了主时钟，其他的时钟源可根据需要选择关闭，以便节省功耗。同样进入 IDLE 模式前，可根据需要设定芯片某些外设的开关，打开的外设在 IDLE 下仍然可以正常工作。

进入 IDLE 模式前，需要先查看一下寄存器 IDLF，如果所有位都为 0，则 CPU 将正常进入 IDLE 模式。如果 IDLF 的位不全为 0，即使有设置进入 IDLE 模式的操作，CPU 也不会进入 IDLE 模式，而是继续停留在正常工作模式。此时用户需先把对应的中断处理完成，再重新设置进入 IDLE 模式。

所有复位事件和任何中断事件都将唤醒芯片。中断唤醒 CPU 后，芯片首先恢复时钟，然后响应中断，进入该中断的服务程序。退出中断服务程序后，芯片将执行置位 IDLE (PCON[0]) 指令后的指令。退出 IDLE 模式时，IDLE 位将自动清零。

需要注意的是，在置位 IDLE (PCON[0]) 的指令后面需要紧接两条 nop 指令，防止程序出错。

4.4.2 STOP 介绍

STOP 模式是比 IDLE 更深层次的低功耗模式。STOP 模式可以停止所有时钟（包括主时钟）和时钟产生电路。如果 WDT 和 WKT 处于打开状态，则它们使用的时钟模块将处于工作状态，可以有选择地关闭 WDT 和 WKT 以节省功耗。

类似于 IDLE 模式，进入 STOP 模式前，需要先查看 STPF 寄存器，若有置 1 的位存在，需要先行处理，以确保能顺利进入 STOP 模式。

STOP 模式可以通过外部中断、LVD 中断或复位、外部硬复位、WKT 中断、WDT 中断或复位、触摸中断来唤醒。如果是中断唤醒，那么唤醒 MCU 后，芯片首先恢复时钟，然后响应该中断，进入该中断的服务程序。退出中断服务程序后，芯片将执行置位 STOP 指令后的指令。退出 STOP 模式时，STOP 位将自动清零。

在进入 STOP 模式时，最后一个时钟沿将关闭系统时钟，然后芯片完全进入 STOP 模式。需要注意的是，在置位 STOP 的指令后面需要紧接两条 nop 指令，防止程序出错。

注意事项：

- 1、IDLF 和 STPF 是只读寄存器，只能读，不能写；

- 2、INT0/Timer0/INT1/Timer1 中断, 进入中断函数会自动清标志;
- 3、芯片进入 STOP 模式, 会关掉系统时钟和 HRC 时钟即 IIC 下载口时钟, 需要加延时复位后才能下载程序;
- 4.想要关闭对用的时钟需要先关闭所有使用本时钟的外设, 如果有一个使用此时钟的外设还没关闭, 则始终无法关闭。

4.4.3 例程代码

进入 IDLE 模式代码例程:

```
/******  
*函数名称: void enterIdle(void)  
*函数功能:  
*入口参数:  
*出口参数:  
*****/  
void enterIdle(void)  
{  
    if(IDLF == 0)  
    {  
        //关掉相关外设  
        PWCON|=0x40;    //Idle 功耗约 458uA  
        // PWCON|=0x00;    //Idle 功耗约 458uA  
        // PWCON|=0x80;    //Idle 功耗约 458uA  
        // PWCON|=0xc0;    //Idle 功耗约 458uA  
  
        HRCON &= 0x7f;  
        LRCON &= 0x7f;  
        MECON |= 0x80;    //IDLE/STOP 模式下, Flash 进入睡眠模式  
  
        PCON = 0x01; //芯片进入 IDLE 模式  
        _nop_();  
        _nop_();  
        _nop_();  
        _nop_();  
        _nop_();  
        //等待唤醒, 此处可以读 IDLF 来查看唤醒标志  
    }  
}
```

进入 STOP 模式代码例程:

```
/******  
*函数名称: void enterStop(void)  
*函数功能:  
*入口参数:  
*出口参数:
```

```
void enterStop(void)
{
    if(STPF == 0)
    {
        //关掉相关外设
        PWCON|=0x40;    //stop 功耗约 2.2uA
        // PWCON|=0x00;    //stop 功耗约 8.7uA
        // PWCON|=0x80;    //stop 功耗约 68.3uA
        // PWCON|=0xc0;    //stop 功耗约 68.3uA

        HRCN &= 0x7f;
        LRCON &= 0x7f;
        MECON |= 0x80;    //IDLE/STOP 模式下, Flash 进入睡眠模式

        PCON |= 0x02; //芯片进入 STOP 模式
        _nop_();
        _nop_();
        _nop_();
        _nop_();
        _nop_();
        //等待唤醒, 此处可以读 STPF 来查看唤醒标志
    }
}
```

测试 IDLE 模式代码例程:(外部中断 0 唤醒, 其他例程可查看 IDLE_STOP.c 文件)

```
/*
*函数名称: void IDLE_Timer0_Test(void)
*函数功能: 外部中断 0, 可唤醒芯片; 外部管脚为 P12
*入口参数: void
*出口参数: void
*/

void IDLE_INT0_Test(void)
{
    unsigned char i=0;

    /* 外部中断 0 */
    P12F = 1;
    IT0CON=0X00|0X0A;    //P12---下降沿或低电平
    TCON|=3;//外部中断 0 在输入管脚下下降沿时触发
    IE=0x81;//开中断

    P17F = 2;
```

```
while(1)
{
    i=20;
    while(i--)
    {
        P17=~P17;
    }

    /* 注意: 外部中断 0 和外部中断 1, 进入中断函数内会自动清标志 */
    enterIdle();//芯片进入 IDLE 模式
}
}
```

STOP 模式代码例程:(外部中断 0 唤醒, 其他例程可查看 IDLE_STOP.c 文件)

```
/******
*函数名称: void STOP_INT0_Test(void)
*函数功能: 外部中断 0, 可唤醒芯片; 外部管脚为 P12
*入口参数: void
*出口参数: void
*****/

void STOP_INT0_Test(void)
{
    unsigned char i=0;

    /* 注意: Stop 模式下, 系统时钟和 HRC 时钟即 IIC 下载口时钟关掉了, 需要加延时复位后
    才能下载程序 */
    i=100;while(i--)delay(100);

    /****** 外部中断 0******/
    P12F = 1;
    IT0CON=0X00|0X0A;    //P12---下降沿或低电平
    TCON|=3;//外部中断 0 在输入管脚下降沿时触发
    IE=0x81;//开中断

    P17F = 2;
    while(1)
    {

        i=20;
        while(i--)
        {
            P17=~P17;
        }
    }
}
```

/* 注意: 外部中断 0 和外部中断 1, 进入中断函数内会自动清标志 */

enterStop();//芯片进入 STOP 模式

}

}

4.5 Timer0/1

4.5.1 Timer0/1 介绍

使用 Timer0 需要配置 TCON, TMOD, TH0, TL0 寄存器配置工作模式, 配置 IE 寄存器的中断, 编写中断处理函数。如果是作为计数器则还需要配置引脚(P34)。

使用 Timer1 需要配置 TCON, TMOD, TH1, TL1 寄存器配置工作模式, 配置 IE 寄存器的中断, 编写中断处理函数。如果是作为计数器则还需要配置引脚(P35)。

Timer0/1 定时器或计数器功能可通过 CT0/CT1 配置选择, CT0=0 选择为定时器, CT0=1 选择为计数器。作为定时器时, 时钟是系统时钟的 12 分频。作为计数器时, 时钟是 T0 端口的输入。定时器 0 有 4 个工作模式, 通过模式选择位 T0M0 和 T0M1 来选择。如下:

Timer0: 模式选择

- (1).TMOD[1:0]=00,模式 0: 定时器 0 作为 13 位定时器/计数器
- (2).TMOD[1:0]=01,模式 1: 定时器 0 作为 16 位定时器/计数器
- (3).TMOD[1:0]=10,模式 2: 定时器 0 作为 8 位自动重载定时器/计数器
- (4).TMOD[1:0]=11,模式 3: TL0 和 TH0 作为两个独立的 8 位定时器/计数器

Timer1: 模式选择

- (1).TMOD[5:4]=00,模式 0: 定时器 1 作为 13 位定时器/计数器
- (2).TMOD[5:4]=01,模式 1: 定时器 1 作为 16 位定时器/计数器
- (3).TMOD[5:4]=10,模式 2: 定时器 1 作为 8 位自动重载定时器/计数器
- (4).TMOD[5:4]=11,模式 3: 此模式下, TH1、TL1 会被锁住, 等效于 TR1=0。

Timer0/1: 管脚配置

- (1).计数功能, 管脚 T0(P34)设置为输入(P34F = 0x1;)
- (2).计数功能, 管脚 T1(P35)设置为输入(P35F = 0x1;)

注意事项: timer0 和 timer1, 进入中断函数会自动清标志。

4.5.2 例程代码

(定时器计时中断 1ms 相应配置)

Timer0-code 如下:

```
/****** Timer0 配置模式 0 函数 *****/
TMOD |= 0x00;           //定时器,时钟为系统时钟 12 分频
                        //模式 0,TL0 和 TH0 组成 13 位定时器/计数器
TL0 = (65536-1334) & 0x1F; //initial timer0 low byte
TH0 = (65536-1334) >> 5;   //initial timer0 high byte(1ms)
ET0 = 1; //开中断 定时器 0 中断使能控制位
```

```
EA = 1;    //总中断
TR0 = 1;   //开始运行//timer0 start running
```

Timer1-code 如下:

```
/****** Timer1 配置模式 0 函数******/
TMOD |= 0x00;           //定时器，时钟为系统时钟 12 分频
                        //模式 0, TL0 和 TH0 组成 13 位定时器/计数器
// TMOD |= CT1;         //计数器和定时器选择位, 1:计数器
TL1 = (65536-1334) & 0x1F; //initial timer0 low byte
TH1 = (65536-1334) >> 5;   //initial timer0 high byte(1ms)
ET1 = 1; //开中断 定时器 1 中断使能控制位
EA = 1; //总中断
TR1 = 1; //开始运行//timer1 start running
```

中断服务函数如下:

```
/*******/
void Timer0_isr (void) interrupt 1
{
    P17=~P17;
    /* set timer0 as mode0 (13-bit) */
    TL0 = (65536-1334) & 0x1F; //initial timer0 low byte
    TH0 = (65536-1334) >> 5;   //initial timer0 high byte
}
void Timer1_isr (void) interrupt 3
{
    P17=~P17;
    /* set timer0 as mode0 (13-bit) */
    TL1 = (65536-1334) & 0x1F; //initial timer0 low byte
    TH1 = (65536-1334) >> 5;   //initial timer0 high byte
}
```

4.6 Timer2

4.6.1 Timer2 介绍

使用 Timer2 需要配置 T2CON, T2MOD, TL2, TH2, T2CL, T2CH 寄存器配置工作模式, 配置 IE 寄存器的中断, 编写中断处理函数。如果是作为计数器则还需要配置引脚(P10 或 P11)。

一、定时器 2 有 4 种工作模式: 重载模式、抓取模式、波特率产生器模式和时钟输出模式, 工作模式描述如下:

(1).重载模式 0: 设置 DCEN = 0, CPRL2 = 0, 如果 EXEN2 = 1, 则 T2EX 边沿和计数溢出时将触发重载, 否则只有计数溢出才能触发重载。

(2).重载模式 1: 设置 DCEN = 1, CPRL2 = 0, 此时 T2EX 为低时定时器递减计数,

T2EX 为高时定时器递增计数。EXF2 标志不再产生中断,而是作为计数器得第 17 位翻转。

(3).抓取模式: 设置 CPRL2 = 1, T2CM = 0, T2EX 管脚边沿触发抓取; 设置 CPRL2 = 1, T2CM = 1, 软件写入寄存器 T2CL 将触发一次抓取操作。

(4).波特率产生器模式: 查看“UART 使用介绍”。(5).时钟输出模式: 设置 T2OE = 1, 此时定时器自动进入计数溢出重载模式 T2 引脚输出频率为 2 倍溢出周期的时钟。此模式不能使用 T2 作为时钟输入。

二、定时器 2 需要注意的管脚配置如下:

(1).计数功能,使用 T2 输入时钟计数, 管脚 T2(P10)设置为输入(P10F = 0x1;), T2MOD.2 可选上/下沿计数。

(2).时钟输出模式, 管脚 T2(P10)设置为输出(P10F = 0x2;)。

(3).T2EX 抓取或触发重载, 管脚 T2EX(P11)设置为输入(P10F = 0x1;), T2MOD.3 可选上/下沿触发重载或抓取。

4.6.2 例程代码

```
/******  
*函数名称: void Timer2Mode0(void)  
*函数功能: 重载模式 0: 可选 T2EX 边沿触发重载  
*入口参数:  
*出口参数:  
*****/  
void InitTimer2_Mode0(void)  
{  
    /* 定时器 2: 重载模式 0 */  
    T2CH = 0x0;    //重载值的高字节  
    T2CL = 0x0;    //重载值的低字节  
    TH2 = T2CH;    //计数值的高字节  
    TL2 = T2CL;    //计数值的低字节  
    T2MOD = 0+0x80+0x08;    //重载模式 0,开中断使能,+0x4/T2 边沿选择,/T2EX 边沿选择  
    T2CON = 0x04+0x08;    //启动模块,/T2EX 使能,+0x02/计数定时选择,+0x1/抓取功能,  
}  
  
/******  
*函数名称: void Timer2Mode1(void)  
*函数功能: 重载模式 1: EXF2 标志,作为计数器得第 17 位翻转  
*入口参数:  
*出口参数:  
*****/  
void InitTimer2_Mode1(void)  
{  
    /* 定时器 2: 重载模式 1 */  
    T2CH = 0x0;  
    T2CL = 0x0;  
    TL2 = T2CL;
```

```
TH2 = T2CH;
T2MOD = 1+0x80;//重载模式 1,开中断使能,+0x4/T2 边沿选择,/T2EX 边沿选择
T2CON = 0+0x04;//启动模块,+0x08/T2EX 使能,+0x02/计数定时选择,+0x1/抓取功能,
}

/*****
*函数名称: void Timer2OutT(void)
*函数功能: (时钟输出信号)
*入口参数:
*出口参数:
*****/
void InitTimer2_OUT(void)
{
    /* 定时器 2: (时钟输出信号) */
    T2CH = 0xff;
    T2CL = 0xff;
    TL2 = T2CL;
    TH2 = T2CH;
    T2MOD = 2+0x80;//时钟输出模式,开中断使能,+0x4/T2 边沿选择,+0x08/T2EX 边沿选择
    T2CON = 0x04;//启动模块,+0x08/T2EX 使能,+0x02/计数定时选择,
}

/*****
*函数名称: void InitTimer2_Capture(void)
*函数功能: (软件写抓取模式)
*入口参数:
*出口参数:
*****/
void InitTimer2_Capture(void)
{
    /* 定时器 2: (软件写抓取模式) */
    T2CH = 0x0;
    T2CL = 0x0;
    TL2 = T2CL;
    TH2 = T2CH;
    T2MOD = 0+0x80+0x10;//重载模式,开中断使能,+0x4/T2 边沿选择,+0x08/T2EX 边沿选择,/软件抓取
    T2CON = 0x04+0x1;//启动模块,+0x08/T2EX 使能,+0x02/计数定时选择,/抓取功能,
}

/*****
*函数名称: void InitTimer2_Trigger(void)
*函数功能: (硬件写抓取模式)
*入口参数:
*出口参数:
```

*****/

```
void InitTimer2_Trigger(void)
{
    /* 定时器 2: (硬件抓取模式) */
    T2CH = 0x0;
    T2CL = 0x0;
    TL2 = T2CL;
    TH2 = T2CH;
    T2MOD = 0+0x80+0x08;//重载模式,开中断使能,+0x4/T2 边沿选择,/T2EX 边沿选择,+0x10/软件抓取
    T2CON = 0x04+0x08+0x1;//启动模块,/T2EX 使能,+0x02/计数定时选择,/抓取功能,
}
```

中断服务函数如下:

```
****重载模式 0*****/
void Timer2_isr (void) interrupt 5
{
    /* 计数溢出标志为 TF2 (T2CON[7]) */
    if(TF2 == 1)
    {
        TF2 = 0;
    }
}

****重载模式 1*****/
void Timer2_isr (void) interrupt 5
{
    /* 计数溢出标志为 TF2 (T2CON[7]) */
    if(TF2 == 1)
    {
        TF2 = 0;
        if(EXF2 == 0)
        {
            P17 = ~P17;//翻转 P17
        }
    }
}

****抓取模式*****/
void Timer2_isr (void) interrupt 5
{
    /* T2EX 的边沿触发标志为 EXF2 (T2CON[6]) */
    if(EXF2 == 1)
    {
        IntFlag = 1;    //可置位标志
    }
}
```

```

        TL2 = 0;
        TH2 = 0;
        EXF2 = 0;
    }
}
    
```

4.7 UART0

UART0 是一个全双工同步/异步串行数据收发器，完全兼容标准 8051，但不同于标准 8051 的是本 UART0 可以设置任意波特率。

4.7.1 UART0 的 4 种工作模式波特率公式

SM00	SM10	模式	描述	波特率
0	0	0	同步移位模式	$F_{sys}/12$
0	1	1	8 位异步模式	波特率= $(2^{SMOD}/32)*$ 定时器 1/2 溢出率
1	0	2	9 位异步模式	当 SMOD=0 时，波特率为 $F_{sys}/64$ 当 SMOD=1 时，波特率为 $F_{sys}/32$
1	1	3	9 位异步模式	波特率= $(2^{SMOD}/32)*$ 定时器 1/2 溢出率

例如，UART0 工作模式 1 或模式 3，计算波特率 $baudrate = (2^{SMOD}/32)*$ 定时器 1/2 溢出率。其中，溢出率=溢出周期的倒数，定时器 1 工作在模式 2（8 位自动重装载）情况下的溢出周期= $(256-初始值)*12/F_{sys}$ 。其中 256 是 8 位定时器溢出计数值，如果定时器 1 工作在模式 1 为 $(0xffff+1)$ ， F_{sys} 是系统时钟。

因此可以使用定时器 1 或者定时器 2 作为溢出率的发生器，波特率如下表：

系统时钟	TH1/TL1 值	定时器 1 计满值	SMOD=0, 波特率	c
16000000	256-1	1	41666	83333
16000000	256-2	2	20833	41666
16000000	256-3	3	13888	27777
16000000	256-4	4	10416	20833
16000000	256-5	5	8333	16666
...	...	...	...	...

系统时钟	TH2/TL2 值	定时器 2 计满值	SMOD=0, 波特率	SMOD=1, 波特率
16000000	65536-10	10	50000	100000
16000000	65536-20	20	25000	50000
16000000	65536-30	30	16667	33333
16000000	65536-40	40	12500	25000
16000000	65536-50	50	10000	20000
16000000	65536-52	52	9615	19231
16000000	65536-60	60	8333	16667
...	...	...	...	...

4.7.2 定时器波特率分频设置(修改分频系数)

由于 WS51F60/70 系列芯片的主时钟最高为 16MHz（或 16MHz 的分频），配置比较高的波特率时有较大误差，因此 UART0 的波特率不再固定为定时器溢出率的 32 分频，而是可以由 S0DIV 设置。

芯片默认的分频系数为 32，与标准 8051 相同。如果要更改分频系数，通过设置 S0DE=1 来使能，S0DIV 的数值表示不同的分频系数，详见寄存器 S0CFG 描述。当 S0DE=1 时，SMOD 位（PCON[7]）将不起作用。

例如:由于系统时钟是固定的，要达到更准确的波特率，只能通过修改分频系数来实现。如果设置定时器 5 个时钟周期溢出，那么： $16000000 / 115200 \times 5 = 27.78$ 。取分频数为 28，那么波特率为 114285，和 115200 相比，误差率约为 0.8%，一般情况下不会影响 UART0 通信。另外，更小的分频数也可以实现更高的波特率配置。如下表：

波特率	分频系数(S0DIV)	Fsys/分频系数/波特率	定时器 2 的时钟周期	误差率约
115200	28	$16000000/28/115200$	5	0.79%
115200	20	$16000000/20/115200$	7	0.79%

注意：实际填到 S0CFG 中的 S0DIV 时，要减一。

4.7.3 参考代码

注意:UART0 的 RX/TX 的 IO 口配置

RX:P30F=6;或 P33F=6;

TX:P31F=6;或 P34F=6;

P33 和 P34 为备选通道，具有较低优先级。如果同时打开首选和备选通道，信号会从首选通道输出，备选通道此时为高阻。

Uart0: 模式选择配置例程代码如下：

//UART0 波特率宏定义

```
#define UART0_BAUDRATE_10417_T1  4  //
#define UART0_BAUDRATE_8334_T1    5  //
#define UART0_BAUDRATE_6945_T1    6  //
#define UART0_BAUDRATE_5952_T1    7  //
#define UART0_BAUDRATE_5208_T1    8  //
#define UART0_BAUDRATE_4800_T2    104 //
#define UART0_BAUDRATE_9600_T2     52 //
#define UART0_BAUDRATE_19200_T2    26 //
#define UART0_BAUDRATE_38400_T2    13 //
#define FASTBAUD_115200_T2         0xff //
```

模式 1 初始化代码如下：

/******

*函数名称： void InitUart0_Timer1(uint8_t baud)

*函数功能: 模式 1(8 位异步模式), Freq-主频 16M, baud-波特率

*入口参数: baud-(UART0_BAUDRATE_5208_T1...)

*出口参数:

*****/

void InitUart0_Timer1(uint8_t baud) //选择 Timer1 作为波特率信号发生器

```
{
    SM00 = 0;
    SM10 = 1; //模式 1(8 位异步模式),
    REN0 = 1; //串行接收使能位
    RI0 = 0; //接收中断标志
    TI0 = 0; //发送中断标志

    //TIMER1 特率时钟
    /* 默认定时器 1 作为时钟源(配置位 8Bit 自动重载模式不开中断程序) */
    /* TL1 作为 8 位定时器/计数器, TH1 作为自动重载寄存器 */
    TR1 = 0; //先停 T1
    TMOD = 0x20; // 定时器, 时钟为系统时钟 12 分频
    TH1 = 256-baud;//10417Hz
    TR1 = 1; //开始运行//timer1 start running
    ES0 = 1;
}
```

*函数名称: void InitUart0_Timer2(uint8_t baud)

*函数功能: 模式 1(8 位异步模式), Fsys-主频 16M, baud-波特率

*入口参数: baud-(UART0_BAUDRATE_4800_T2,...)

*出口参数:

*****/

void InitUart0_Timer2(uint8_t baud) //选择 Timer2 作为波特率信号发生器

```
{
    SM00 = 0;
    SM10 = 1; //模式 1(8 位异步模式),
    REN0 = 1; //串行接收使能位
    RI0 = 0; //接收中断标志
    TI0 = 0; //发送中断标志

    if(FASTBAUD_115200_T2 == baud) //FASTBAUD_115200_T2 = 0xff
    { //快速波特率设置(修改分频系数)
        baud = 7;
        S0CFG = (0x01<<7)+(20-1); // S0DE=1 S0DIV=20
    }

    //TIMER2 特率时钟
    TR2 = 0;
    T2MOD = 0x00; /* 定时 2 中断不使能, 不分频 */
    T2CL = 65536 - baud;
```

```
T2CH = 65536 - baud >> 8;
TL2 = T2CL;
TH2 = T2CH;
TR2 = 1;    //启动定时器 2
RCLK = 1;
TCLK = 1;
ES0 = 1;
}
```

发送例程代码如下:

```
/******
*函数名称: void UART0_write(signed char dat)
*函数功能: UART0_write
*入口参数: void
*出口参数: void
*****/

void UART0_write(signed char dat)
{
    S0BUF = dat;    //Send data to UART buffer
    while(!TI0);
    TI0 = 0;
}
```

中断服务函数如下:

```
/******Uart0 中断函数*****
void Uart_Isr(void) interrupt 4
{
    if(RI0 == 1)    //如果接收到计算机发的数据
    {
        RI0 = 0;    //清除标志位
    }
}
```

4.8 PWM

WS51F60/70 系列芯片 PWM 介绍: 最多有 6 路独立 PWM 输出, 可组成 3 组 (PWM0/2/4)互补 PWM 输出, 在 6 路独立 PWM 输出模式下周期和占空比可在 16 位范围内任意配置, 在 3 组互补 PWM 输出模式下支持 8 位死区控制。

芯片直接输出 HRC 会出现时钟抖动, 如果是检查芯片 HRC 时钟, 请对 HRC 进行分频后输出, 或者在芯片 VDD 和 GND 之间接一个 0.1UF 或以上的电容。

```
/******
```

使用问题说明:

1、PWM 有备选通道

2、**60/70 系列存在使用问题描述:** PWM 独立输出模式下, 如果要使用 PWM1(3/5)需

要打开 PWM0(2/4)和 PWM1(3/5)两个的使能，只开 PWM1(3/5)/PWM0(2/4)的使能用不了

*****/

4.8.1 PWM 工作原理

这里以 PWM0/1 为例，配置 PWM：

当 PWMnSN 为 1 时，PWM0 和 PWM1 可以有各自的频率和占空比。配置 PWMEN 使能 PWM，可选 PWMIE 开启或不开启中断，PWM0CFG 配置 PWM0 和 PWM1 的输出模式，PWM0DIVL 和 PWM0DIVH 设置 PWM0 频率，PWM0DUTL 和 PWM0DUTH 设置 PWM0 占空比，PWM1DIVL 和 PWM1DIVH 设置 PWM1 频率，PWM1DUTL 和 PWM1DUTH 设置 PWM1 占空比。

当 PWMnSN 为 0 时，PWM0 和 PWM1 组成互补 PWM 输出，频率由 PWM0DIVL 和 PWM0DIVH 控制，占空比由 PWM0DUTL 和 PWM0DUTH 控制，死区时间由 PWM0DTS（与 PWM1DUTL 同一地址）控制。配置 PWMEN 使能 PWM，可选 PWMIE 开启或不开启中断，PWM0CFG 配置 PWM0 和 PWM1 的输出模式。PWM0DIVL 和 PWM0DIVH 设置 PWM0 频率，PWM0DUTL 和 PWM0DUTH 设置 PWM0 占空比，PWM0DTS（与 PWM1DUTL 同一地址）设置死区时间。PWM1 根据 PWM0CFG 的设置得到与 PWM0 同向或者反向的波形。波形见数据手册的“插入死区时间的 PWM0/1 互补波形配置示意图”。

其他两组 PWM 与 PWM0、PWM1 同理。

注意：每个 PWMn 输出 IO 口，都有一个对应备选 IO 口[PWMn]，可同时输出相同波形。（查看芯片规格资料 IO 端口）

公式如下：PWM_CLK 为 PWM 选择的时钟源，可选为 HRC,LRC 或者系统时钟；

- * 频率 = $\text{PWM_CLK}/(\text{PWM0DIVH} \times 256 + \text{PWM0DIVL})$
- * 占空比 = $\text{PWMnDUT}/(\text{PWMnDIV} + 1)$;
- * 输出高低电平：PWMnDUT=0, PWMnDIV!=0 极性对应 PWMnOP 或 PWMnON;
- * 输出时钟源： PWMnDIV=0;
- * 死区周期： PWMSNn=0, 则 PWMnDUTL 八位寄存器为 PWM(n-1)DTS[7:0]，用于 PWM0/2/4 的死区时间控制; n=1,3,5

4.8.2 例程代码

初始化例程代码如下：

*****/

*函数名称：void init_PWM01(void)

*函数功能：配置为互补输出

*入口参数：

*出口参数：

*****/

void init_PWM01(void)

{//功能描述: PWM0 频率 32KHz, 占空比 50%, 死区周期 3.125us, 时钟源 HRC-16M

P10F=0X04; // IO 配置为 PWM0

P11F=0X04; // IO 配置为 PWM1

```
//P17F=0X04; // IO 配置为[PWM0],备选口
//P33F=0X04; // IO 配置为[PWM1],备选口
PWM0CFG=0x02; //时钟源 HRC, PWM0/1 为互补输出, 即 PWM1 作 PWM0N
//PWM0CFG=0x12; //时钟源 HRC, PWM0/1 独立输出, 即 PWM1 作 PWM1

PWM0DUTL=250%256;
PWM0DUTH=250/256; //占空比为 250/501 = 50%
PWM0DIVL=500%256;
PWM0DIVH=500/256; //频率为 16000000/500 = 32KHz
PWM1DUTL=50; //死区周期为 50*(1/16M) = 3.125us

PWMEN |= 0x01; //使能 PWM0 模块
PWMIE |= 0x01; //使能 PWM0 中断
}

/*****
*函数名称: void init_PWM01(void)
*函数功能: 配置为独立输出
*入口参数:
*出口参数:
*****/
void init_PWM01(void)
{//功能描述: PWM0/1 频率 32KHz,占空比 50%,时钟源 HRC-16M
    P10F=0X04; // IO 配置为 PWM0
    P11F=0X04; // IO 配置为 PWM1

    //P17F=0X04; // IO 配置为[PWM0],备选口
    //P33F=0X04; // IO 配置为[PWM1],备选口
    //PWM0CFG=0x02; //时钟源 HRC, PWM0/1 为互补输出, 即 PWM1 作 PWM0N
    PWM0CFG=0x12; //时钟源 HRC, PWM0/1 独立输出, 即 PWM1 作 PWM1

    PWM0DUTL=250%256;
    PWM0DUTH=250/256; //占空比为 250/501 = 50%
    PWM0DIVL=500%256;
    PWM0DIVH=500/256; //频率为 16000000/500 = 32KHz
    PWM1DUTL=250%256;
    PWM1DUTH=250/256; //占空比为 250/501 = 50%
    PWM1DIVL=500%256;
    PWM1DIVH=500/256; //频率为 16000000/500 = 32KHz

    PWMEN |= 0x03; //使能 PWM0/1 模块
    PWMIE |= 0x01; //使能 PWM0 中断
}
```

中断服务函数如下:

```
/******PWM 中断函数******/  
void INT3_isr(void) interrupt 5  
{  
    if(PWMFLG & 0x01)//judge interrupt flag  
    {  
        //PWM0 标志  
        PWMFLG &= 0xFE;//clear interrupt flag  
    }  
}
```

4.9 I2C

4.9.1 I2C 介绍

I2C 接口支持芯片与外围 I2C 器件以标准 I2C 协议进行串行数据传输,其主要特点如下

- * 支持主机或从机模式,可自动切换角色
- * 7 位从机地址
- * 支持多主机仲裁
- * 支持广播功能
- * 支持标准/快速/高速模式,速度分别达到 100K / 400K /3M bits/s
- * 对系统时钟没有要求,在任何系统时钟 I2C 接口都可正常通讯

4.9.2 I2C 通信原理

标准的 I2C 通信由四部分组成: START 信号、从机地址传输、数据传输和 STOP 信号,数据传输是以 START 信号开始,以 STOP 信号结束。如图 6.3.3-1 所示, I2C 主机发送 START 信号开启 I2C 通信,然后主机发送从机地址寻址从机,接收到从机 ACK 信号后,开始发送或接收数据,在全部数据传送结束后,由主机发送 STOP 信号,结束通信。

4.9.3 I2C 工作模式

I2C 接口有 3 种工作模式: 等待模式、从机模式、主机模式。

- * 默认情况下, I2C 处于等待模式。
- * I2C 主动发送 START 信号,从等待模式切换到主机模式。
- * 在等待模式时,如果接受到 START 信号,则 I2C 从等待模式切换到从机模式。

主机和从机都 hold, 主机和从机时钟 16M, SCL 速度可到 2M; 从机不开 hold 大概 200kHz-220kHz(中断只清除标志)。

寄存器介绍:

I2CTXD----发送数据寄存器, 存放待发送数据
 I2CRXD----接收数据寄存器, 存放接收到数据
 I2CFLG----状态标志寄存器,可查询总线忙空闲状态,总线接收响应 ACK,总线仲裁,Start/Stop 信号,RX/TX 标志,从机接收地址标志
 I2CCON----控制寄存器, 使能和选择分频时钟, 发送 Start/Stop 等
 I2CADR----从机地址寄存器, 作为从机有效
 I2CFG0----配置寄存器, hold 选择,掩膜 I2C 接收到的从机地址, 跟 7 位地址一一对应

/*****

注意事项:

- 1、P30/P31 是备选通道, 优先级比较低, 管脚开漏输出要加外部上拉, 或打开内部上拉
- 2、主机发送 Stop 信号前面会发一个数据(I2CTXD)再 stop;
- 3、stop 后发送 Start, 要加一点延时
- 4、主机模式 hold 默认开 (主机模式:且关不掉)
- 5、WS51F60/70 系列主从模式都不支持混合模式(restart)
- 6、从机模式:I2C 作为从机正确接收到从机地址,自动回 ACK

*****/

4.9.4 例程代码作为主机

/*****

流程:

(默认情况下,I2C 处于等待模式.发送 START 信号,切换到主机模式)

主机初始化:

- 1、P30/P31 配置成 SCL/SDA;
- 2、I2C 时钟频率配置;
- 3、模块中断使能配置
- 4、hold 可配置也可不配置(主机模式默认开)

主机写数据:

- 1、发送写设备地址 dev_val: 填写设备地址 dev_val,发送 start 信号
- 2、发送寄存器地址(即 datS[0]): 检查发送成功标志 tx,成功填写 datS[0]到 I2CTXD,清 tx 标志;失败退出
- 3、写数据 datS[]: 检查发送成功标志 tx,成功继续填写 datS 到 I2CTXD,清 tx 标志;失败退出
- 4、直到发送完 datS[],发送 Stop 信号

主机读数据:

- 1、发送写设备地址 dev_val: 填写设备地址 dev_val,发送 start 信号
- 2、发送寄存器地址 0x00: 检查发送成功标志 tx,成功填写 0x00 到 I2CTXD,清 tx 标志;失败退出
- 3、发送 Stop 信号, 填写从机设备地址读(dev_val+1),发送 Start 信号
- 4、读数据 datR[]: 检查发送成功标志 rx,成功保存 I2CRXD 到 datR,清 rx 标志;失败退出
- 5、直到读取完 datR[],发送 Stop 信号

*****/

```
//寄存器 I2CFLG 标志位
#define BUSIDLE 0x80          //总线状态标志（只读）
#define RXNAK 0x40           //总线接收响应标志（只读）
#define IF_LSTARB 0x20       //I2C 发送的数据和接收到的数据不一致时出现总线丢失，高有效，写 0 清
#define IF_RXSTA 0x10        //总线接收到 START 信号
#define IF_RXSTP 0x08        //总线接收到 STOP 信号
#define IF_TXDAT 0x04        //当 I2C 为主机时表明已发送一个字节数据并接收到响应标志
#define IF_RXDAT 0x02        //接收一个字节数据完成
#define IF_RXADR 0x01        //I2C 作为从机正确接收到从机地址
```

```
/******
```

```
*函数名称: void InitHoseI2C(void)
```

```
*函数功能: I2C 初始化:主机模式, 配置时钟 14.8K, 不开中断使能, 默认开 hold
```

```
*管脚 SCL:P30, SDA:P31
```

```
*入口参数: void
```

```
*出口参数: void
```

```
*****/
```

```
void InitHoseI2C(void)
```

```
{
    P33F = 0;//
    P34F = 0;//
    P30F =0xa5;//IIC SCL
    P31F =0xa5;//IIC SDA

    I2CCON = 0x03; //Fi2c_CLK 时钟分频选择-----> 0:16M, 1:16M/2, 2:16M/4, 3:16M/8
    I2CFG1 = 0xff; //I2C 数据输出频率=Fi2c/(I2CFG1<6:0>+8);即:16M/8/(0x7F+8) = 14.8K

    I2CFLG = 0x0;          //清标志
    I2CCON |= (0x1<<7);    //MODE en
    I2CCON &= ~(0x1<<6);  //不开中断使能

    I2CTXD = 0x0;          //I2C 发送数据寄存器
    I2CCON &= ~(0x1<<5);  //START INT EN 不开中断使能
    I2CCON &= ~(0x1<<4);  //STOP INT EN 不开中断使能

    I2CFG0 |= (0x1<<7); //hold on 主机模式 hold 默认开（主机模式:且关不掉）
}
```

```
/******
```

```
*函数名称: void I2C_WriteByteNum(uint8_t device_addr, uint8_t *dat, uint8_t length)
```

```
*函数功能: IIC 写数据
```

```
*入口参数:
```

```
device_addr: 设备地址
```

```
dat: 存放写缓存数组
```

length: 写字节个数

*出口参数:

*****/

uint8_t I2C_WriteByteNum(uint8_t device_addr, uint8_t *dat, uint8_t length)

```
{
    uint8_t ret=0, cnt=0;
    uint16_t tim=0;

    I2CFLG = 0; //请标志
    I2CTXD  = device_addr;    //0x00 填写设备(device_addr);
    I2CCON  |= (0x1<<3);    //IIC_Start();

    while(1)
    {
        if(IF_RXSTA == (I2CFLG&IF_RXSTA))    //start
        {
            I2CFLG &= ~IF_RXSTA;
        }

        if(IF_TXDAT == (I2CFLG&IF_TXDAT))
        {
            I2CTXD  = dat[cnt];    //
            cnt++;

            //发送完退出
            if(cnt >= length)
            {
                I2CCON  |= 0x1<<2;    //STOP IIC
                I2CFLG &= (~IF_TXDAT);
                I2CFLG = 0;
                return 0;
            }

            I2CFLG &= (~IF_TXDAT);

            //超时/出错退出
            if(RXNAK == (I2CFLG&RXNAK))
            {
                ret = 2;
                I2CCON  |= 0x1<<2;    //STOP IIC

                I2CFLG = 0;
                return ret;
            }
        }
    }
}
```

```
}
//仲裁丢失
if(IF_LSTARB == (I2CFLG&IF_LSTARB))
{
    ret=1;
    I2CFLG &= ~IF_LSTARB;
    I2CFLG = 0;
    return ret;
}
}
}

/*****
*函数名称: void I2C_ReadByteNum(uint8_t addr,uint8_t device_addr, uint8_t *dat, uint8_t num)
*函数功能: IIC 读数据
*入口参数:
    addr: 寄存器地址
    device_addr: 设备地址
    dat: 存放读缓存数组
    num: 读取字节个数
*出口参数:
*****/
uint8_t I2C_ReadByteNum(uint8_t addr,uint8_t device_addr, uint8_t *dat, uint8_t num)
{
    uint8_t ack=0,step=0,ret=0;
    uint16_t tim=0;

    I2CFLG = 0; //请标志

    I2CTXD = device_addr; //写 从机设备地址(device_addr);
    I2CCON |= (0x1<<3); //发送起始信号 start

    while(1)
    {
        if(IF_TXDAT == (I2CFLG&IF_TXDAT))//tx 发送完成标志
        {
            if(step == 0)
            {
                I2CTXD = addr; //寄存器地址
                I2CCON |= 0x1<<2; //STOP IIC
                step=1;
            }
        }
    }
}
```

```
//从机回复 Nak, 主机发送 stop 退出
if(RXNAK == (I2CFLG&RXNAK))
{
    ret=2;
    I2CCON |= 0x1<<2; //STOP IIC
    I2CFLG = 0;
    return ret;
}

I2CFLG &= (~IF_TXDAT);
}

if(IF_RXSTP == (I2CFLG&IF_RXSTP)) //stop 标志
{
    if(step==1)
    {
        //STOP 发送后,需要加一点延时,才能发送
        delay_us(348); //IIC 使用 14K 左右时钟(延时约 0.2ms 以上即可)
        //delay_us(6); //IIC 使用 2M 左右时钟(延时约 2us 以上即可)
        //delay_us(40);
        I2CTXD = device_addr+1; //读 从机设备地址(device_addr+1);
        I2CCON |= (0x1<<3); //start tx
    }
    I2CFLG &= ~IF_RXSTP;
}

if(IF_RXDAT == (I2CFLG&IF_RXDAT)) //rx_flag 发送完成标志
{
    *dat = I2CRXD;

    if((--num) == 0)
    {
        I2CCON |= 0x1<<2; //STOP IIC
        I2CFLG &= ~IF_RXDAT;
        return 0;
    }
    dat++;
    I2CFLG &= ~IF_RXDAT;
}

//仲裁丢失
if(IF_LSTARB == (I2CFLG&IF_LSTARB))
{
    ret = 1;
}
```

```
I2CFLG &= ~IF_LSTARB;

return ret;    //1
}

}

}

}

/*****
*函数名称: void I2C_HoseTest(void)
*函数功能: I2C 主机读写测试
*入口参数: void
*出口参数: void
*****/
#define HOSE_SEND_NUM 3
#define HOSE_RECEIVE_NUM 3
void I2C_HoseTest(void)
{
    int i;
    unsigned char dev_val = 0;
    uint8_t datS[HOSE_SEND_NUM]={0};
    uint8_t datR[HOSE_RECEIVE_NUM]={0};

    SCCON = 0x00;//选择高速时钟 HRC
    HRCON |= 0x80;
    i=100;while(i--)>0;delay(200); //如果使用了下载口 P33/P34，需要在 mian 函数中初始化 P33/34 之前
    加延时，复位后才能下载程序

    /***** IIC 初始化配置 *****/
    InitHoseI2C();
    //test CODE
    dev_val = 0xA0; //主机发送设备地址 0xA0

    // 主机发送数据缓存 赋值
    for(i=0;i<HOSE_SEND_NUM;i++)
    {
        datS[i]=i+0x30; //0123.....
    }

    for(i=0;i<HOSE_RECEIVE_NUM;i++)
    {
        datR[i]=0; //清数组
    }

    while(1)
    {
```

```

        if(BUSIDLE == (I2CFLG&BUSIDLE))    //总线空闲
        {
            I2C_WriteByteNum(dev_val,datS,HOSE_SEND_NUM);
            delay(100);//STOP 发送后,需要加一点延时,才能发送
            I2C_ReadByteNum(0x00,dev_val,datR,HOSE_RECEIVE_NUM);
            delay(100);//STOP 发送后,需要加一点延时,才能发送
        }
        delay(500);
    }
}

```

4.9.5 例程代码作为从机

```

/*****
流程:
(在等待模式时, 如果接受到 START 信号, 则 I2C 从等待模式切换到从机模式)
从机初始化:
1、P30/P31 配置成 SCL/SDA;
2、从机地址设置;
3、模块中断使能
4、hold 开启
从机中断函数:
1、判断主机读写
2、主机读,从机发送数据: 填写 I2CTXD,清标志,检查发送成功标志 tx,成功填写 I2CTXD,清 tx 标志;
3、主机写,从机接收数据: 检查接收成功标志 rx,成功保存 I2CRXD 到 dat,清 rx 标志;
*****/

/*****
*函数名称: void InitSlaveI2C(void)
*函数功能: I2C 初始化:从机模式, 开中断使能, 开 hold
*
*管脚 SCL:P30, SDA:P31
*入口参数: void
*出口参数: void
*****/

void InitSlaveI2C(void)
{
    P33F = 0;//
    P34F = 0;//
    P30F = 5;//IIC SCL
    P31F = 5;//IIC SDA

    /* 1、从机接收到设备地址, 产生标志 IF_RXADR, 有没有 IF_RXDAT 标志? 没有
    * 2、从机: 回 nak, 不清 rx 标志, (操作-hold, SDA, SCL 一直为低; 关掉 hold, 不清标志回 nak)
    * 3、从机: 回 ak, 清 rx 标志
    */

    // I2CADDR = 0x80; //识别广播地址 0

```

```
I2CADDR = 0x55;    //从设备地址设置，第 7 位广播不能使能
```

```
I2CCON = 0x00; //I2c_CLK
```

```
I2CFG0 |= (0x1<<7); //HOSE on
```

```
// I2CFG0 &= ~(0x1<<7); //HOSE not
```

```
I2CFLG = 0x0;
```

```
I2CCON |= (0x1<<7); //mode en
```

```
I2CCON |= (0x1<<6);
```

```
I2CTXD = 0x0; //0x00
```

```
I2CCON |= (0x1<<5); //START INT EN
```

```
I2CCON |= (0x1<<4); //STOP INT EN
```

```
}
```

中断服务函数如下:

```
#define SLAVE_SEND_NUM 20
```

```
#define SLAVE_RECEIVE_NUM 20 //最大接收缓存
```

```
uint8_t reveice_cnt=0,rx_flag=0,send_cnt=0;
```

```
uint8_t datS[SLAVE_SEND_NUM];
```

```
uint8_t datR[SLAVE_RECEIVE_NUM];
```

```
/******I2C 中断函数******/
```

```
void i2c_trdata(void) interrupt 5
```

```
{
```

```
if(IF_RXADR == (I2CFLG&IF_RXADR)) //adr
```

```
{
```

```
if(I2CRXD&0x01 == 0x01) //host----read
```

```
{
```

```
send_cnt = 0;
```

```
I2CTXD = datS[send_cnt];
```

```
send_cnt++;
```

```
}
```

```
reveice_cnt = 0; //清空接收数据计数
```

```
I2CFLG &= ~IF_RXADR;
```

```
}
```

```
else if(IF_RXDAT == (I2CFLG&IF_RXDAT)) //slave-----RX
```

```
{
```

```
if(reveice_cnt>=SLAVE_RECEIVE_NUM) //接收数据超过缓存
```

```
{
```

```
reveice_cnt=0;
```

```
}
```

```
datR[reveice_cnt] = I2CRXD;
```

```
reveice_cnt++;
```

```
I2CFLG &= ~IF_RXDAT;    //暂停总线，等清 RX 标志，释放总线，回 ack;
}

else if(IF_TXDAT == (I2CFLG&IF_TXDAT))    //slave-----TX
{
    I2CTXD = datS[send_cnt];

    if(++send_cnt>=SLAVE_SEND_NUM)
    {
        send_cnt=0;
    }
    I2CFLG &= ~IF_TXDAT;    //暂停总线，等清 TX 标志，释放总线;
}
else if(IF_RXSTP == (I2CFLG&IF_RXSTP))    //stop
{
    if(reveice_cnt!=0)
        rx_flag = 1;
    I2CFLG &= ~IF_RXSTP;
}
}
}

/******
*函数名称: void I2C_SlaveTest(void)
*函数功能: I2C 从机读写测试
*入口参数: void
*出口参数: void
*****/

void I2C_SlaveTest(void)
{
    int i;

    SCCON = 0x00;//选择高速时钟 HRC
    HRCON |= 0x80; //
    i=150;while(i--)>0;delay(200);

    InitSlaveI2C();    //I2C 寄存器配置
    EINT2 = 1;
    EA = 1;            //开中断使能

    // 赋值从机 要发送的数据缓存
    for(i=0;i<SLAVE_SEND_NUM;i++)
    {
        datS[i] = i+0x65;//ABCD...
    }

    for(i=0;i<SLAVE_RECEIVE_NUM;i++)
```

```
{  
    datR[i]=0; //清数组  
}  
//test CODE slave  
while(1);  
}
```

4.10 WDT

4.10.1 WDT 介绍

看门狗定时器（WDT）为一个 15 位递减计数器，其计数时钟为 LRC 时钟，有 8 位调节精度，计数范围为 7.8125ms-1s。

4.10.2 例程代码

```
/**  
*函数名称: void WDT_Test(void)  
*函数功能: WDT 测试(溢出时间 1s, 开中断, WDT 中断翻转 P17, 清标志)  
*入口参数: void  
*出口参数: void  
***/  
void WDT_Test(void)  
{  
  
    WDTFLG=0x5a;//close wdt  
  
    //WDT 初始化  
    WDTCON=0X87;    //溢出中断使能,时间选择 1s  
    IE = 0xC0;    //global 和 WDT 中断开  
  
    WDTFLG=0XA5;//open wdt  
  
    P17F=2;  
    P30F=0X02;  
    P17=0;  
  
    while(1){  
        P30=~P30;  
        //    WDTFLG=0XA5;//喂狗  
    }  
}
```

中断服务函数如下:

```
/** *****WDT 中断函数***** */  
void int5() interrupt 6  
{  
    if((WDTFLG&0x02) == 0x02)  
    {  
        P17=~P17;  
        WDTFLG=0XA5;  
    }  
}
```

4.11 WKT

4.11.1 WKT 介绍

- 1、芯片内置了一个自唤醒定时器（WKT），用于低功耗模式下的周期唤醒芯片，也可用作通用定时器。
- 2、WKT 固定使用内部 32K 时钟源 LRC，如果开启 WKT 计数，则 LRC 会自动开启。
- 3、通过 WKTSS 可选每 0.5 或 1 秒产生一次中断。

4.11.2 例程代码

```
/** ***** */  
*函数名称: void WKT_Test(void)  
*函数功能: WKT 测试:中断选择 1s, 开中断, 中断翻转 P17, 清标志  
*入口参数:  
*出口参数:  
/** ***** */  
void WKT_Test(void)  
{  
    WKTCON = 0xC0;    //模块使能,中断选择 1s  
  
    IE = 0xC0;        //global 和 WKT 中断开  
  
    P17F=2;  
    P30F=0X02;  
    P17=0;  
    while(1)  
    {  
        P30=~P30;  
    }  
}
```

中断服务函数如下:

```
/******WKT 中断函数******/  
void int5() interrupt 6  
{  
    if((WKTCON&0x01) == 0x01)  
    {  
        P17=~P17;  
        WKTCON &= 0XFE;  
    }  
}
```

4.12 DAC

4.12.1 DAC 介绍

WS51F60/70 系列 集成了一个 5 位可编程数模转换器 (DAC)。DAC 的输入参考可以选择为来自芯片外部电源、内部电压基准或者外部端口 DAVE 输入。

4.12.2 功能描述

DAC 模块通过 31 个大小相等的电阻串联形成均匀的 32 级分压,其电压源可选外部电源 VDD、内部运放输出的 1.5V 电压或外部 DAVE 端口。使用时通过 DAVS 选择电压大小,该电压经过一个 buffer 输出到外部端口 DACO,可驱动 3mA 的电流。

注意:选择输出内部 1.5V 基准电压时芯片无法输出;

4.12.3 例程代码

```
/****** DAC 配置函数******/  
P34F = 0x03;    //DAC 输出 IO 复用配置  
DACON = 0x80+31; //选择外部电源 VDD 为参考; DA 输出电压为 (基准电压*DAVS/31)
```

4.13 LVD

4.13.1 简介

低电压检测 (LVD) 用于监控芯片自身的供电电压 VDD,可设置检测电压范围为 1.8V-4.2V,精度±50mV。内含去抖功能。当 VDD 小于所设定的电压值时,可设置触发中断或复位。

4.13.2 功能描述

配置寄存器 LVDS 不为 0 时开启 LVD 功能，其触发电压阈值由寄存器 LVDTH 设置。

当芯片 VDD 出现由高变低到 LVDTH 设置的阈值时，LVDF 位将置 1。此时，如果 LVDS 为 1 则产生 LVD 中断，如果 LVDS 为 2 则产生 LVD 复位，如果 LVDS 为 3 则芯片即不复位也不产生中断，此时通过 LVDF 标志可初步判断芯片电源电压。如果产生了 LVD 复位，不会复位寄存器 LVDCON。

4.13.3 例程代码

```
/****** LVD 配置函数******/  
LVRCON = 0x40; //低压时产生中断， 低压阈值设置 1.8V  
IE = 0xC0;      //global 和 LVD 中断开
```

中断服务函数如下：

```
/****** LVD 中断函数******/  
void int5() interrupt 6  
{  
    if((LVRCON & 0x01) == 0x01)  
    {  
        LVRCON = LVRCON & 0xfe;  
    }  
}
```

4.14 软件 LED 驱动

4.14.1 软件 LED 介绍

WS51F60/70 系列 系列芯片内置了一个软件 LED 驱动，该驱动与触摸按键模块共用 12 个引脚。这 12 个引脚可以任意连接到 LED 的阴极或者阳极，软件配置对应的寄存器即可。如果电路板设计为共阳数码管，更改为共阴数码管时不需要修改电路和程序，只需要修改 LSCFG 这个寄存器即可。

4.14.2 LED 显示原理

每一个 LED 管脚都可设置为 com 或 seg，且点亮和熄灭时间可以通过寄存器 LSDTM 与 LSPTM 设置，点亮时间越长，则 LED 越亮。其计算公式如下：

LED 显示时间 = LSDTM * LRC 时钟周期(≈32KHz，即≈31.25us)

LED 熄灭时间 = LSPTM * LRC 时钟周期(≈32KHz，即≈31.25us)

备注： LSDTM 与 LSPTM 的和不要小于 2，否则无法产生中断标志

4.14.3 LED 与 Touch Key 共用方法

LED 与 Touch Key 可共用引脚并同时使用，其实现方法为在 LED 熄灭时间软件开启触摸按键扫描功能。即将 Touch Key 配置为手动模式，当 LED 点亮结束时，开启 Touch Key 扫描。另外用户可通过 LTF (LSCON[0])标志查看 Touch Key 扫描的时间是否合适。

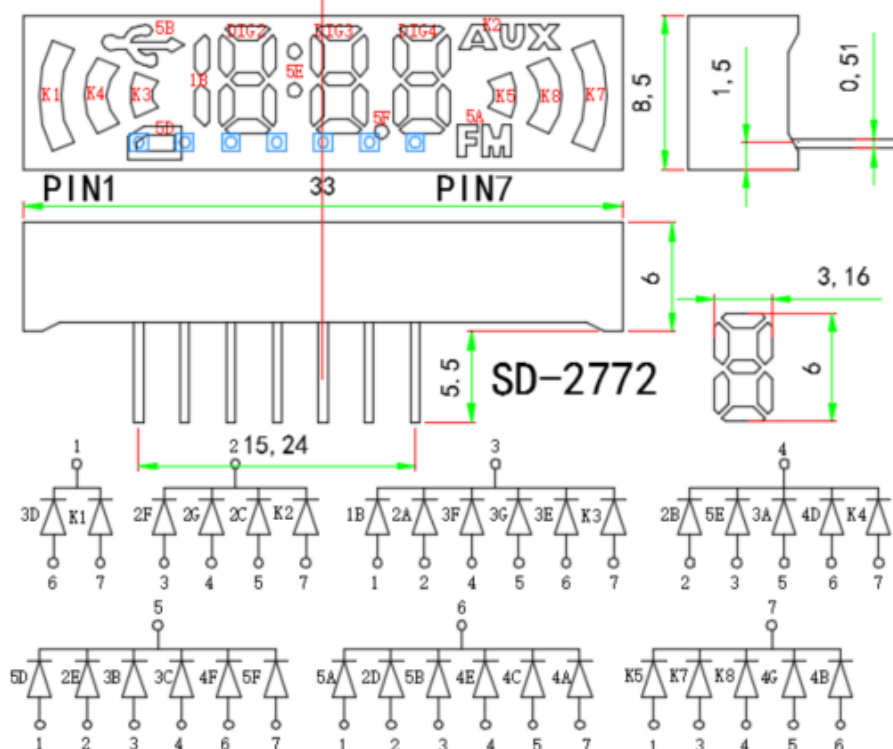
轮询 LED 数码管使用介绍，数码管图片如下：

配置流程：

- 1、IO 可以配置成 PnxF = 0;或 PnxF = 3;且使能寄存器 LSPE 对应的 com/seg 引脚
- 2、配置寄存器 LSCFG, 选择 com 和 seg 点亮和熄灭阶段时的有效电平;
- 3、配置寄存器 LSDTM/LSPTM, 设置点亮熄灭时间
- 4、配置寄存器 LSCON, 使能 LED 模块/中断
- 5、选择 com 脚(配置寄存器 LSCOM),填写 seg 值(配置寄存器 LSDAT),
- 6、扫描完所有 com 完成一帧 LED 图像

注意事项：

- 1、seg 值(LSDAT): 0 高阻状态, 1 对应 LSCFG 配置
- 2、同一时间只有一个 COM,其它都是 seg 口



4.14.4 轮询 LED 例程代码

```
#define DIG2(0)
```

```
#define DIG3(1)
#define DIG4(2)
// 芯片内置了一个软件 LED 驱动，可以用来驱动矩阵 led(com/seg 共用)，原理图看同目录下的
“SD2772.png” 文件
// MCU 与 LED 接线如下
// P10--1
// P11--2
// P12--3
// P13--4
// P14--5
// P15--6
// P36--7
// 字符 ‘0’ - ‘9’ 的逻辑段码
// D7 D6 D5 D4 D3 D2 D1 D0
// 段 A B C D E F G DOT
unsigned char code seg_code[]={~0x03, ~0x9f, ~0x25, ~0x0d, ~0x99, ~0x49, ~0x41, ~0x1f, ~0x01, ~0x09,
0x00};
unsigned char seg_buf[7]; // led 段码缓存
unsigned char display_buf[3]; // 3 位数字[2 3 4]对应的显示缓存
unsigned char led_k; // k1 k2 k3 k4 k5 k7 k8 显示缓存
unsigned char led_5; // 5A 5B 5D 5E 5F 的显示缓存
unsigned char led_1b; // 1b 显示缓存

/*****
*函数名称: void seg2seg(void)
*函数功能: 更新 seg_buf[7], led 段码缓存
*入口参数:
*出口参数:
*****/
void seg2seg(void)
{
    unsigned char seg_tmp;
    unsigned char i;

    // seg_buf[0]
    seg_buf[0] = 0; // 共阴极接法，0 表示不亮
    if (led_k & 0x01) // K1
        seg_buf[0] |= (0x01<<7);
    i = display_buf[DIG3];
    seg_tmp = seg_code[i];
    if (seg_tmp & 0x10) // 3D
        seg_buf[0] |= (0x01<<6);
    seg_buf[0] = seg_buf[0]>>1; // 由于 led 原理图下标从 1 开始
```

```
// seg_buf[1]
seg_buf[1] = 0;
if (led_k & 0x02)           // K2
    seg_buf[1] |= (0x01<<7);
i = display_buf[DIG2];
seg_tmp = seg_code[i];
if (seg_tmp & 0x20)         // 2C
    seg_buf[1] |= (0x01<<5);
if (seg_tmp & 0x02)         // 2G
    seg_buf[1] |= (0x01<<4);
if (seg_tmp & 0x04)         // 2F
    seg_buf[1] |= (0x01<<3);
seg_buf[1] = seg_buf[1]>>1;

// seg_buf[2]
seg_buf[2] = 0;
if (led_k & 0x04)           // K3
    seg_buf[2] |= (0x01<<7);
i = display_buf[DIG3];
seg_tmp = seg_code[i];
if (seg_tmp & 0x08)         // 3E
    seg_buf[2] |= (0x01<<6);
if (seg_tmp & 0x02)         // 3G
    seg_buf[2] |= (0x01<<5);
if (seg_tmp & 0x04)         // 3F
    seg_buf[2] |= (0x01<<4);
i = display_buf[DIG2];
seg_tmp = seg_code[i];
if (seg_tmp & 0x80)         // 2A
    seg_buf[2] |= (0x01<<2);
if (led_1b)                 // 1B
    seg_buf[2] |= (0x01<<1);
seg_buf[2] = seg_buf[2]>>1;

// seg_buf[3]
seg_buf[3] = 0;
if (led_k & 0x08)           // K4
    seg_buf[3] |= (0x01<<7);
i = display_buf[DIG4];
seg_tmp = seg_code[i];
if (seg_tmp & 0x10)         // 4D
    seg_buf[3] |= (0x01<<6);
i = display_buf[DIG3];
seg_tmp = seg_code[i];
```

```
if (seg_tmp & 0x80)          // 3A
    seg_buf[3] |= (0x01<<5);
if (led_5 & 0x08 )          // 5E
    seg_buf[3] |= (0x01<<3);
i = display_buf[DIG2];
seg_tmp = seg_code[i];
if (seg_tmp & 0x40)          // 2B
    seg_buf[3] |= (0x01<<2);
seg_buf[3] = seg_buf[3]>>1;

// seg_buf[4]
seg_buf[4] = 0;
if (led_5 & 0x04)           // 5F
    seg_buf[4] |= (0x01<<7);
i = display_buf[DIG4];
seg_tmp = seg_code[i];
if (seg_tmp & 0x04)          // 4F
    seg_buf[4] |= (0x01<<6);
i = display_buf[DIG3];
seg_tmp = seg_code[i];
if (seg_tmp & 0x20)          // 3C
    seg_buf[4] |= (0x01<<4);
if (seg_tmp & 0x40)          // 3B
    seg_buf[4] |= (0x01<<3);
i = display_buf[DIG2];
seg_tmp = seg_code[i];
if (seg_tmp & 0x08)          // 2E
    seg_buf[4] |= (0x01<<2);
if (led_5 & 0x10)           // 5D
    seg_buf[4] |= (0x01<<1);
seg_buf[4] = seg_buf[4]>>1;

// seg_buf[5]
seg_buf[5] = 0;
i = display_buf[DIG4];
seg_tmp = seg_code[i];
if (seg_tmp & 0x80)          // 4A
    seg_buf[5] |= (0x01<<7);
if (seg_tmp & 0x20)          // 4C
    seg_buf[5] |= (0x01<<5);
if (seg_tmp & 0x08)          // 4E
    seg_buf[5] |= (0x01<<4);
if (led_5 & 0x40)           // 5B
    seg_buf[5] |= (0x01<<3);
```

```
i = display_buf[DIG2];
seg_tmp = seg_code[i];
if (seg_tmp & 0x10)          // 2D
    seg_buf[5] |= (0x01<<2);
if (led_5 & 0x80)           // 5A
    seg_buf[5] |= (0x01<<1);
seg_buf[5] = seg_buf[5]>>1;

// seg_buf[6]
seg_buf[6] = 0;
i = display_buf[DIG4];
seg_tmp = seg_code[i];
if (seg_tmp & 0x40)          // 4B
    seg_buf[6] |= (0x01<<6);
if (seg_tmp & 0x02)          // 4G
    seg_buf[6] |= (0x01<<5);
if (led_k & 0x80)            // K8
    seg_buf[6] |= (0x01<<4);
if (led_k & 0x40)            // K7
    seg_buf[6] |= (0x01<<3);
if (led_k & 0x10)            // K5
    seg_buf[6] |= (0x01<<1);
seg_buf[6] = seg_buf[6]>>1;
}
```

/******

*函数名称: void init_sled(void)

*函数功能: 初始化 IO 和 led 寄存器配置

*入口参数:

*出口参数:

*****/

```
void init_sled(void)
```

```
{
    P10F = 0;    //高阻/LED0 seg0
    P11F = 0;    //高阻/LED1 seg1
    P12F = 0;    //高阻/LED2 seg2
    P13F = 0;    //高阻/LED3 seg3
    P14F = 0;    //高阻/LED4 seg4
    P15F = 0;    //高阻/LED5 seg5
    P36F = 0;    //高阻/LED6 seg6
    LSPE0 = 0x7f; //COM/SEG 的使能 7 个
    LSPE1 = 0x00;
```

```
    LSCFG = 0x02; // LED 点亮阶段, SEG/COM 有效电平选择 [SEG=H COM=L]
```

```
// LED 显示时间 = LSDTM * LRC 时钟周期( $\approx 32\text{KHz}$ , 即  $\approx 31.25\mu\text{s}$ )
```

```
LSDTM0 = 35;
```

```
LSDTM1 = 0;          // LED 显示时间设置
```

```
// LED 熄灭时间 = LSPTM * LRC 时钟周期( $\approx 32\text{KHz}$ , 即  $\approx 31.25\mu\text{s}$ ) 注意: LSDTM 与 LSPTM  
的和不要小于 2, 否则无法产生中断标志
```

```
LSPTM0 = 35;
```

```
LSPTM1 = 0;          // LED 熄灭时间设置
```

```
LSCON = 0xc0; // 点亮结束中断
```

```
IE = 0xa0;
```

```
}
```

中断服务函数如下:

```
/******中断函数******/
```

```
void SEG_isr() interrupt 5
```

```
{
```

```
    static unsigned char i=0;
```

```
    LSCON &= 0xF0;    // 清中断标志
```

```
    LSDAT0 = seg_buf[i];
```

```
    LSCOM=i;
```

```
    i++;
```

```
    if(i==7)
```

```
    {
```

```
        i=0;
```

```
        seg2seg(); // 显示完一帧后, 更新显示内容
```

```
    }
```

```
}
```

主函数如下:

```
void delay_1ms(void)
```

```
{
```

```
    int j;
```

```
    for(j=0;j<1700;++j);
```

```
}
```

```
void main()
```

```
{
```

```
    unsigned int nTick;
```

```
    unsigned char i;
```

```
    SCCON=0x00;
```

```
HRCON|=0x80; //
```

```
init_sled();
display_buf[DIG2] = 8;
display_buf[DIG3] = 8;
display_buf[DIG4] = 8;
led_1b = 0xff;
led_k = 0xff;
led_5 = 0xff;
seg2seg();
nTick = 0;
i=0;
while(1)
{
    delay_1ms();
    nTick++;
    if (nTick>1000)
    {
        nTick = 0;

        display_buf[DIG2]++;
        if (display_buf[DIG2]>9)
            display_buf[DIG2] = 0;

        display_buf[DIG3]++;
        if (display_buf[DIG3]>9)
            display_buf[DIG3] = 0;

        display_buf[DIG4]++;
        if (display_buf[DIG4]>9)
            display_buf[DIG4] = 0;

        led_1b = ~led_1b;

        led_5 = (0x01<<i);

        if (i == 0)
            led_k = (0x01<<1);          // 点亮 k2
        else if (i == 1)
            led_k = (0x01<<1)+(0x01<<2)+(0x01<<4); // 点亮 k2 k3 k5
        else if (i == 2)
            led_k = (0x01<<1)+(0x01<<2)+(0x01<<4)+(0x01<<3)+(0x01<<7); // 点亮 k2 k3
            k5 k4 k8
        else if (i == 3)
```

```

    led_k
    (0x01<<1)+(0x01<<2)+(0x01<<4)+(0x01<<3)+(0x01<<7)+(0x01<<0)+(0x01<<6);// 点亮 k2 k3 k5 k4 k8 k1 k7
    else
        led_k = 0x00;

    i++;
    if (i>7)
        i=0;
    }
  }
}

```

4.14.5 共阳数码管例程代码

```

unsigned char code smg[]={
    0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,
    0x77,0x7c,0x39,0x5e,0x79,0x71};//共阴数码管显示字符 0-F
/*****
*函数名称: void InitLED(void)
*函数功能: LED 初始化
*入口参数: void
*出口参数: void
*****/
void InitLED(void)
{
    /* 下面 12 个 IO 配置为 LED 引脚 */
    P10F = 3;  //seg
    P11F = 3;  //seg
    P12F = 3;  //seg
    P13F = 3;  //seg
    P14F = 3;  //seg
    P15F = 3;  //seg
    P16F = 3;  //seg
    P17F = 3;  //seg
    P30F = 3;  //com
    P31F = 3;  //com
    P32F = 3;  //com
    i=150;while(i--)>0;delay(100);
    P33F = 3;  //com
    LSPE0 = 0xFF;
    LSPE1 = 0x0f;//COM 和 seg 的使能
    LSCFG = 0x01;  //配置点亮阶段,seg/com 有效电平
    LSDTM0 = 125;
    LSDTM1 = 0;//点亮时间配置

```

```
LSPTM0 = 0;
LSPTM1 = 0X00; //熄灭时间
LSCON = 0xC0;
```

```
LSDAT0 = 0x6F;
LSDAT1 = 0x00; //配置 seg 值
LSCOM = 0x08; //选择 com8
EINT2 = 1;
EA = 1;
```

```
}
```

中断服务函数如下:

```
/** LED 中断函数 */
void SEG_isr() interrupt 5
{
    LSCON &= 0xF0; //清标志
    if(LSCOM == 0x0B)
    {
        LSCOM = 0x08;
    }
    else
        LSCOM++; //切换 com(8,9,10,11)
    LSDAT0 = smg[LSCOM&0xF]; //填写 seg 值
}
```

4.15 软件 LCD 驱动

4.15.1 软件 LCD 描述

芯片的 P10-P14 可作为软件 LCD 的 COM 口,这些 IO 可作为普通 IO 功能之外,还可输出 $1/2$ VDD 电压。因此用户可通过软件实现点亮 LCD 屏的功能。
详情查看芯片规格书 6.11 描述。

4.15.2 例程代码

例程代码可查看 lcd.c

```
// 芯片内置了一个软件 LCD 驱动, 可以驱动 1/2 偏置的 LCD。
// 字符 '0'-'9'的逻辑段码
// 位    D7 D6 D5 D4 D3 D2 D1 D0
// 段    A  B  C  D  E  F  G  DP
#include "WS51F60/70 系列.h"
unsigned char code seg_code[10]={~0x03, ~0x9f, ~0x25, ~0x0d, ~0x99, ~0x49, ~0x41, ~0x1f, ~0x01,
~0x09};
```

```
unsigned char seg_buf[3];           // lcd 的段码缓存
unsigned char display_buf[3];       // 3 位数字对应的显示缓存
unsigned char scan_step;             // 扫描计数
// LCD 的管脚定义与 LED 不一样，不是每个 COM 对应一位数字所有 SEG，而是对应每一位数字的
一部分 SEG。
```

```
// 1 2   3   4 5 6 7 8 9 10
// COM1           1F 1B 1A 2A 3A 3F 3B
//           COM2   1G 1C 2F 2G 2B 3G 3C
//           COM3 1E 1D 2E 2D 2C 3E 3D
// MCU   LCD
// P10---1 COM1
// P11---2 COM2
// P12---3 COM3
// P13---4 SEG0
// P14---5 SEG1
// P15---6 SEG2
// P16---7 SEG3
// P17---8 SEG4
// P30---9 SEG5
// P31---10 SEG6
```

```
/******
```

```
*函数名称: void seg2seg(void)
```

```
*函数功能: 更新 seg_buf[3], lcd 段码缓存
```

```
*入口参数:
```

```
*出口参数:
```

```
*****/
```

```
void seg2seg(void)
```

```
{
```

```
    unsigned char seg_tmp;
```

```
    unsigned char dat;
```

```
    seg_buf[0] = 0;
```

```
    seg_buf[1] = 0;
```

```
    seg_buf[2] = 0;
```

```
    dat = display_buf[0];
```

```
    seg_tmp = seg_code[dat];           // 第一位数字
```

```
    if (seg_tmp & 0x80)                 // 段 A
```

```
        seg_buf[0] |= (0x01<<2);
```

```
    if (seg_tmp & 0x40)                 // 段 B
```

```
        seg_buf[0] |= (0x01<<1);
```

```
if (seg_tmp & 0x20)          // 段 C
    seg_buf[1] |= (0x01<<1);

if (seg_tmp & 0x10)          // 段 D
    seg_buf[2] |= (0x01<<1);

if (seg_tmp & 0x08)          // 段 E
    seg_buf[2] |= (0x01<<0);

if (seg_tmp & 0x04)          // 段 F
    seg_buf[0] |= (0x01<<0);

if (seg_tmp & 0x02)          // 段 G
    seg_buf[1] |= (0x01<<0);

dat = display_buf[1];
seg_tmp = seg_code[dat];      // 第二位数字
if (seg_tmp & 0x80)          // 段 A
    seg_buf[0] |= (0x01<<3);

if (seg_tmp & 0x40)          // 段 B
    seg_buf[1] |= (0x01<<4);

if (seg_tmp & 0x20)          // 段 C
    seg_buf[2] |= (0x01<<4);

if (seg_tmp & 0x10)          // 段 D
    seg_buf[2] |= (0x01<<3);

if (seg_tmp & 0x08)          // 段 E
    seg_buf[2] |= (0x01<<2);

if (seg_tmp & 0x04)          // 段 F
    seg_buf[1] |= (0x01<<2);

if (seg_tmp & 0x02)          // 段 G
    seg_buf[1] |= (0x01<<3);

dat = display_buf[2];
seg_tmp = seg_code[dat];      // 第三位数字
if (seg_tmp & 0x80)          // 段 A
    seg_buf[0] |= (0x01<<4);

if (seg_tmp & 0x40)          // 段 B
```

```
seg_buf[0] |= (0x01<<6);

if (seg_tmp & 0x20)      // 段 C
    seg_buf[1] |= (0x01<<6);

if (seg_tmp & 0x10)      // 段 D
    seg_buf[2] |= (0x01<<6);

if (seg_tmp & 0x08)      // 段 E
    seg_buf[2] |= (0x01<<5);

if (seg_tmp & 0x04)      // 段 F
    seg_buf[0] |= (0x01<<5);

if (seg_tmp & 0x02)      // 段 G
    seg_buf[1] |= (0x01<<5);
}

void scan_lcd(void)
{

    switch (scan_step)
    {
        case 0:          // COM1 V+
            PCOMS = 0x06;
            P1 = seg_buf[0]<<3;          //P10 = 0;
            P3 = (seg_buf[0]>>5);
            break;

        case 1:          // COM1 V-
            P1 = ((~seg_buf[0])<<3)+1;    //P10 = 1;
            P3 = ((~seg_buf[0])>>5);
            break;

        case 2:          // COM2 V+
            PCOMS = 0x05;
            P1 = seg_buf[1]<<3;          //P11 = 0;
            P3 = (seg_buf[1]>>5)&0x03;
            break;

        case 3:          // COM2 V-
            P1 = ((~seg_buf[1])<<3)+2;    //P11 = 1;
            P3 = ((~seg_buf[1])>>5);
            break;
```

```
case 4:          // COM3 V+
PCOMS = 0x03;
P1 = seg_buf[2]<<3;          //P12 = 0;
P3 = (seg_buf[2]>>5);
break;

case 5:          // COM3 V-
P1 = ((~seg_buf[2])<<3)+4;    //P12 = 1;
P3 = ((~seg_buf[2])>>5);
break;

default:
break;
}
scan_step++;
if (scan_step>5)
{
    scan_step = 0; // 一帧显示完毕，更新显示内容
    seg2seg();
}
}

void init_lcd(void)
{
    P10F=0x02;    // com1
    P11F=0x02;    // com2
    P12F=0x02;    // com3

    P13F=0x02;    // seg0
    P14F=0x02;    // seg1
    P15F=0x02;    // seg2
    P16F=0x02;    // seg3
    P17F=0x02;    // seg4

    P30F=0x02;    // seg5
    P31F=0x02;    // seg6

    PCOMS=0x07;
    PDRVS=0x01;

    scan_step = 0;
```

```
}
```

```
void delay_1ms(void)
```

```
{
```

```
    int j;
```

```
    for(j=0;j<1700;++j);
```

```
}
```

```
void main(void)
```

```
{
```

```
    int i;
```

```
    unsigned char nTick;
```

```
    SCCON=0x00;
```

```
    HRCN|=0x80; //
```

i=50;while(i--)&delay_1ms(); // 如果使用了下载口 P33/P34 , 需要在 mian 函数中初始化 P33/34 之前加延时, 复位后才能下载程序

```
    init_lcd();
```

```
    nTick=0;
```

```
    display_buf[0] = 0;
```

```
    display_buf[1] = 0;
```

```
    display_buf[2] = 0;
```

```
    seg2seg();
```

```
    while(1)
```

```
    {
```

```
        delay_1ms();
```

```
        scan_lcd();
```

```
        nTick++;
```

```
        if (nTick>200)
```

```
        {
```

```
            nTick = 0;
```

```
            display_buf[0]++;
```

```
            if (display_buf[0]>9)
```

```
                display_buf[0] = 0;
```

```
            display_buf[1]++;
```

```
            if (display_buf[1]>9)
```

```
                display_buf[1] = 0;
```

```
            display_buf[2]++;
```

```
            if (display_buf[2]>9)
```

```
        display_buf[2] = 0;
    }
}
}
```

4.16 Flash 数据区

注意：不建议使用 FLASH 区存储数据，可以优先考虑 NVR1 区存储数据。

FLASH 区默认是用来存储代码的。C 语言里面的 CODE 数组也是存储在 FLASH 里面。函数形参的字符串也是存储在 FLASH 里面。程序存储的位置从 0 地址开始，存储的长度是 keilc51 编译代码的长度。此外，除了存储程序的 flash 外，其他 FLASH 可以提供给用户使用。

Flash 的操作是先解锁，擦除再写入。Flash 又分为 DATA 区和 CODE 区。CODE 区默认是存储程序的。FLASH 掉电后仍然能保存数据。

4.16.1 Flash 区介绍

Flash 数据(DATA)区逻辑划分：

Flash 数据区由若干个扇区组成，扇区是进行擦除操作的最小单位，每个扇区大小为 128 字节。

Flash 区可以通过寄存器 PADR（默认值 PADR=40H，全为程序区）按功能划分为 CODE 区和 DATA 区，CODE 区用于存储用户的程序，DATA 区是用于存储一些掉电需要保存的数据。划分单位为 1 个逻辑扇区。

注意事项：（默认值 BTMSK=0xff;）

划分程序区的地址空间为：0 - (PADR×128 - 1);

数据区的地址空间为：(PADR×128) - 1FFFF;

对 flash 进行数据操作命令时，需先设置寄存器 PADR 值，PADR 不能设置为 0，且 PADR 的值需大于（自己代码长度-1）/128+1，代码长度可在 keil 编译时查看，划分数据区的空间地址。然后可对数据区进行读写擦命令起始扇区值 0 到（0x40-PADR-1）。

譬如 FLASH 容量是 8K，如果代码长度为 129B 则代码长度占了两个扇区，因此 PADR 最少要设置为 02H，不能设置为 01H。在 PADR=0X02H 的情况下，DATA 区域的起始地址为 FLASH 的 256 位置，即 FLASH 的 256 位置为 DATA 区逻辑地址为 0 的位置。以此类推，PADR 为 0X10H 时，FLASH 的 2K 地址为 DATA 区逻辑地址为 0 的位置。

以下为 Flash 区 code/data 基本读写擦函数

```
#define SECTOR_SIZE 128    //扇区大小为 128 字节
#define FLASH_SIZE (8*1024) //芯片 flash 容量
#define INNER_FLASH_CHECK_SUM_CODE 0x0c //当前扇区的累加和命令
#define INNER_FLASH_CHECK_SUM_DATA 0x08 //当前扇区的累加和命令
#define INNER_FLASH_UNLOCK 0X12 //flash 解锁
#define INNER_FLASH_CODE_UNLOCK 0X34 //flash code 区解锁
#define INNER_FLASH_DATA_UNLOCK 0X56 //flash data 区解锁
#define INNER_FLASH_WRITE_DATA 0X02 //写
#define INNER_FLASH_WRITE_CODE 0X06
```

```
#define INNER_FLASH_READ_CODE 0X05    //读
#define INNER_FLASH_READ_DATA 0X01
#define INNER_FLASH_ERASE_CODE 0X07    //擦
#define INNER_FLASH_ERASE_DATA 0X03
#define INNER_FLASH_LOCK 0XFF          //flash 上锁
#define INNER_FLASH_CODE_LOCK 0X43//flash code 区上锁
#define INNER_FLASH_DATA_LOCK 0X65//flash data 区上锁
```

4.16.2 Code 区擦除函数

```
/**
 * @brief   erase innner code area
 * @param   SectorNum: the nume of sector , value is 0 to 8*1024/128
 * @param   len: how many sector need to erase
 * @retval  no
 */
void InnerFlashCodeErase(unsigned short SectorNum,unsigned short len)
{
    unsigned int i=0;
    unsigned short StartAddr;
    //1   Flash unlock
    FSFLG=INNER_FLASH_UNLOCK;
    //2   Flash CODE unlock
    FSFLG=INNER_FLASH_CODE_UNLOCK;
    //3   write address
    StartAddr=SECTOR_SIZE*SectorNum;
    for(i=0;i<len;++i)
    {
        FSADRL=((StartAddr+i*SECTOR_SIZE)&0xff);
        FSADRH((((StartAddr+i*SECTOR_SIZE)>>8)&0xff);
        //4 Write CMD
        FSCMD=INNER_FLASH_ERASE_CODE;
    }
    //5 CODE lock
    FSFLG=INNER_FLASH_CODE_LOCK;
    //6 Flash lock
    FSFLG=INNER_FLASH_LOCK;
}
```

4.16.3 Code 区写数据函数

```
/**
 * @brief   对 FLASH 写入数据，调用本函数写入数据前需要先擦除扇区
 * @param   pBuffer, 要写入数据的指针
```

```
* @param WriteAddr, 写入地址
* @param NumByteToWrite, 写入数据长度
* @retval 无
*/
void InnerFLASHCodeBufferWrite(unsigned char* pBuffer, unsigned short WriteAddr, unsigned short NumByteToWrite)
{
    unsigned int i=0;
    //1 Flash unlock
    FSFLG=INNER_FLASH_UNLOCK;
    //2 Flash CODE unlock
    FSFLG=INNER_FLASH_CODE_UNLOCK;
    FSCMD=INNER_FLASH_WRITE_CODE;
    for(i=0;i<NumByteToWrite;++i)
    {
        FSADRL=((WriteAddr+i)&0xff);
        FSADRH((((WriteAddr+i)>>8)&0xff);
        //4 Write data
        FSDAT=pBuffer[i];
    }
    //5 CODE lock
    FSFLG=INNER_FLASH_CODE_LOCK;
    //6 Flash lock
    FSFLG=INNER_FLASH_LOCK;
}
```

4.16.4 Code 区读数据函数

```
/**
* @brief 读取 FLASH 数据
* @param pBuffer, 存储读出数据的指针
* @param ReadAddr, 读取地址
* @param NumByteToRead, 读取数据长度
* @retval 无
*/
void InnerFLASHCodeBufferRead(unsigned char* pBuffer, unsigned short ReadAddr, unsigned short NumByteToRead)
{
    unsigned int i=0;

    //1 Flash code read
    FSCMD=INNER_FLASH_READ_CODE;
    for(i=0;i<NumByteToRead;++i)
    {
        FSADRL=((ReadAddr+i)&0xff);
```

```
        FSADRH=((ReadAddr+i)>>8)&0xff);  
        //2 read data  
        pBuffer[i]=FSDAT;  
    }  
}
```

4.16.5 Data 区擦除函数

```
/**  
 * @brief   erase innner code area  
 * @param   SectorNum: the nume of sector , value is 0 to 32*1024/128  
 * @param   len: how many sector need to erase  
 * @retval  no  
 */  
void InnerFlashDataErase(unsigned short SectorNum,unsigned short len)  
{  
    unsigned int i=0;  
    unsigned short StartAddr;  
    //1   Flash unlock  
    FSFLG=INNER_FLASH_UNLOCK;  
    //2   Flash DATA unlock  
    FSFLG=INNER_FLASH_DATA_UNLOCK;  
    //3   write address  
    StartAddr=SECTOR_SIZE*SectorNum;  
    for(i=0;i<len;++i)  
    {  
        FSADRL=((StartAddr+i*SECTOR_SIZE)&0xff);  
        FSADRH((((StartAddr+i*SECTOR_SIZE)>>8)&0xff);  
        //4 Write CMD  
        FSCMD=INNER_FLASH_ERASE_DATA;  
    }  
    //5 DATA lock  
    FSFLG=INNER_FLASH_DATA_LOCK;  
    //6 Flash lock  
    FSFLG=INNER_FLASH_LOCK;  
}
```

4.16.6 Data 区写数据函数

```
/**  
 * @brief   对 FLASH 写入数据，调用本函数写入数据前需要先擦除扇区  
 * @param   pBuffer, 要写入数据的指针  
 * @param   WriteAddr, 写入地址  
 * @param   NumByteToWrite, 写入数据长度
```

```

* @retval 无
*/
void InnerFLASHDataBufferWrite(unsigned char* pBuffer, unsigned short WriteAddr, unsigned short NumByteToWrite)
{
    unsigned int i=0;
    //1 Flash unlock
    FSFLG=INNER_FLASH_UNLOCK;
    //2 Flash CODE unlock
    FSFLG=INNER_FLASH_DATA_UNLOCK;

    FSCMD=INNER_FLASH_WRITE_DATA;
    for(i=0;i<NumByteToWrite;++i)
    {
        FSADRL=((WriteAddr+i)&0xff);
        FSADRH((((WriteAddr+i)>>8)&0xff));
        //4 Write data
        FSDAT=pBuffer[i];
    }
    //5 CODE lock
    FSFLG=INNER_FLASH_DATA_LOCK;
    //6 Flash lock
    FSFLG=INNER_FLASH_LOCK;
}

```

4.16.7 Data 区读数据函数

```

/**
* @brief 读取 FLASH 数据
* @param pBuffer, 存储读出数据的指针
* @param ReadAddr, 读取地址
* @param NumByteToRead, 读取数据长度
* @retval 无
*/
void InnerFLASHDataBufferRead(unsigned char* pBuffer, unsigned short ReadAddr, unsigned short NumByteToRead)
{
    unsigned int i=0;
    //1 Flash code read
    FSCMD=INNER_FLASH_READ_DATA;
    for(i=0;i<NumByteToRead;++i)
    {
        FSADRL=((ReadAddr+i)&0xff);
        FSADRH((((ReadAddr+i)>>8)&0xff));
        //2 read data
        pBuffer[i]=FSDAT;
    }
}

```

```
}
```

```
}
```

4.16.8 查看当前 CODE 扇区累加和函数

```
FSADRL=((i*SECTOR_SIZE)&0xff);  
FSADRH((((i*SECTOR_SIZE)>>8)&0xff); //填写扇区地址(当 PADR=0x40 时, i=0~0x3f)  
FSCMD=INNER_FLASH_CHECK_SUM_CODE; //0x0c  
printf("%bx%bx", CKSUMH,CKSUML); //打印当前扇区累加和寄存器值
```

4.16.9 查看当前 DATA 扇区累加和函数

```
FSADRL=((i*SECTOR_SIZE)&0xff);  
FSADRH((((i*SECTOR_SIZE)>>8)&0xff); //填写扇区地址(当 PADR=0x20 时, i=0~0x1f)  
FSCMD=INNER_FLASH_CHECK_SUM_DATA; //0x08  
printf("%bx%bx", CKSUMH,CKSUML); //打印当前扇区累加和寄存器值
```

4.17 NVR 数据区

4.17.1 NVR 数据区简介

芯片内置了 256B 的 NVR 数据区, 该区共 2 个扇区, 每个扇区 128 字节。NVR 的操作跟 FLASH 操作过程一致, 只是发送的命名不同, NVR 掉电后仍然能保存数据。

- (1) NVR1 保留给用户使用, 这是一个不会因为烧写程序而擦写的区域。
- (2) NVR2 可用于实现用户自定义功能, 如 ID 功能, 程序读保护, P32 默认功能。

详情查看芯片规格书 5.2.2.1 描述。

4.17.2 NVR 例程代码

NVR 基本读写擦操作例程代码如下:

```
#define SECTOR_SIZE 128  
#define NVR1_START_ADDR 0X00  
#define NVR2_START_ADDR 0X80  
#define NVR_UNLOCK 0X78  
#define NVR_WRITE 0X82  
#define NVR_ERASE 0X83  
#define NVR_LOCK 0X87  
#define NVR_READ 0X81  
/*****  
  
*函数名称: void EraseNVR(unsigned short SectorAddr)  
*函数功能: NVR 扇区擦除  
*入口参数: SectorAddr_扇区首地址(NVR1/2_START_ADDR)
```

*出口参数: void

*****/

void EraseNVR(unsigned short SectorAddr)

```
{
    //1  Flash unlock
    FSFLG=NVR_UNLOCK;

    //2  write address
    FSADRL=(SectorAddr&0xff);
    FSADRH=((SectorAddr>>8)&0xff);
    //3  Flash erase cmd
    FSCMD=NVR_ERASE;
    //4 NVR lock
    FSFLG=NVR_LOCK;
}
```

*函数名称: void WriteDataNVR(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

*函数功能: NVR 扇区写

*入口参数: buf,要写入数据的指针

SectorAddr,扇区地址

len,写入长度

*出口参数: void

*****/

void WriteDataNVR(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

```
{
    unsigned char i=0;
    //1  Flash unlock
    FSFLG=NVR_UNLOCK;
    //2  Flash  write cmd
    FSCMD=NVR_WRITE;
    for(i=0;i<len;++i)
    {
        //3  write address
        FSADRL=(SectorAddr&0xff);
        FSADRH=((SectorAddr>>8)&0xff);
        SectorAddr++;
        //4 write data
        FSDAT=buf[i];
    }
    //4 NVR lock
    FSFLG=NVR_LOCK;
}
```

*函数名称: void readDatafromNVR(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

*函数功能: NVR 扇区读

*入口参数: buf,要保存数据的缓存

SectorAddr,扇区地址

len,写入长度

*出口参数: void

*****/

void readDatafromNVR(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

```
{
    unsigned char i=0;
    //1 read cmd
    FSCMD=NVR_READ;
    for(i=0;i<len;++i)
    {
        //2 write address
        FSADRL=(SectorAddr&0xff);
        FSADRH=((SectorAddr>>8)&0xff);
        ++SectorAddr;
        //3 read data
        buf[i]=FSDAT;
    }
}
```

4.18 EEPROM 数据区

4.18.1 EEPROM 数据区介绍

芯片内置了 128B 的 EEPROM 数据区,读写该区不需要擦除可直接改写。EEPROM 掉电后仍然能保存数据。NVR 与 EEPROM 相比,NVR 的速度要快一点,EEPROM 写入数据的时间要久一点。EEPROM 数据区操作流程请查看芯片规格书。

4.18.2 例程代码

EEPROM 基本读写操作例程代码如下:

```
#define INNER_EPPROM_UNLOCK 0X9A
```

```
#define INNER_EPPROM_WRITE_CMD 0X42
```

```
#define INNER_EPPROM_LOCK 0XA9
```

```
#define INNER_EPPROM_READ_CMD 0X41
```

```
#define INNER_EEPROM_START_ADDR 0x00
```

```
*****/
```

*函数名称: void writeData2EEPROM(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

*函数功能: EEPROM 写

*入口参数: buf,要写入数据的指针

SectorAddr 地址

len,写入长度

*出口参数: void

*****/

void writeData2EEPROM(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

```
{
    unsigned char i=0;
    //1 EPPROM Unlock
    FSFLG=INNER_EPPROM_UNLOCK;
    //2 EEPROM Enable write
    FSCMD=INNER_EPPROM_WRITE_CMD;
    for(i=0;i<len;++i)
    {
        //3 write address
        FSADRL=(SectorAddr&0xff);
        FSADRH=((SectorAddr>>8)&0xff);
        SectorAddr++;
        //4 write data
        FSDAT=buf[i];
    }
    //4 NVR lock
    FSFLG=INNER_EPPROM_LOCK;
}
```

*****/

*函数名称: void readDatafromEEPROM(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

*函数功能: EEPROM 写

*入口参数: buf,要读出数据的指针

SectorAddr 地址

len,读出长度

*出口参数: void

*****/

void readDatafromEEPROM(unsigned char *buf,unsigned short SectorAddr,unsigned short len)

```
{
    unsigned char i=0;
    //1 EEPROM Enable read
    //FSFLG=INNER_EPPROM_UNLOCK;
    FSCMD=INNER_EPPROM_READ_CMD;
    for(i=0;i<len;++i)
    {
        //2 write address
        FSADRL=(SectorAddr&0xff);
        FSADRH=((SectorAddr>>8)&0xff);
        SectorAddr++;
        //3 write data
    }
```

```
buf[i]=FSDAT;
```

```
}
```

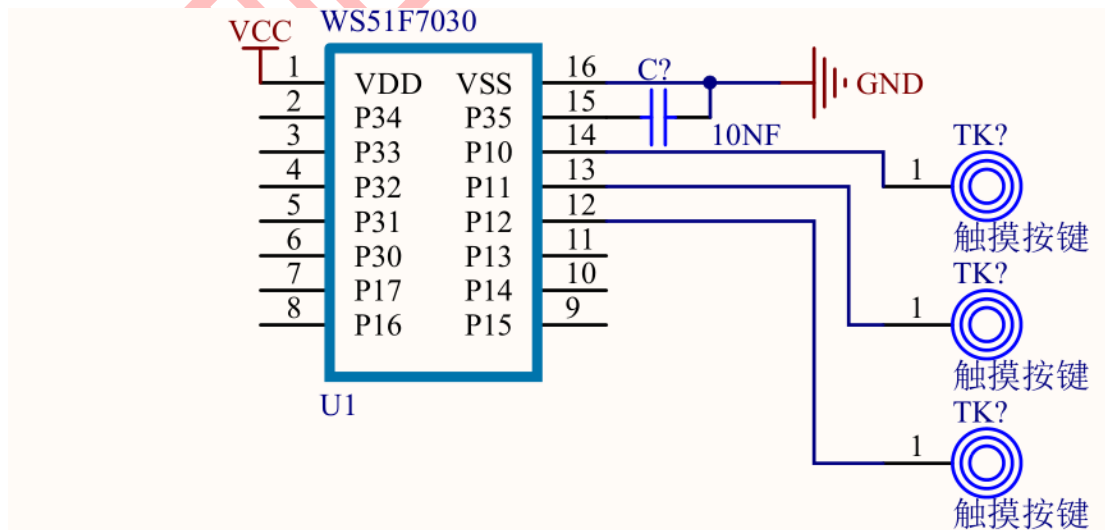
```
}
```

4.19 触摸按键介绍

芯片内置了最大 12 通道的电荷迁移触摸按键模块，它具有优越的抗干扰性能和低功耗特性，可通过 EFT 4KV、CS-10V 等测试，每增加一个按键芯片平均功耗增加 0.3uA，同时内置的比较功能可有效减少采样计算次数，从而减少软件代码量。应用时在 CMOD 端口外接一个 10nF-47nF 电容，电容精度 10%以内，建议使用涤纶电容、X7R 材质电容或 NPO 材质贴片电容。该电容可直接影响触摸灵敏度，电容容值越小，灵敏度越低，容值越大，灵敏度越高。

4.19.1 触摸按键使用

一、应用电路如下：



二、软件配置

为了加快触摸调试开发效率，我们提供了一套调试触摸的开发工具，具体有触摸库代码程序，联调触摸库参数的上位机，还有 WS-LINK (下载器)。

其中，触摸库提供的配置文件 tk_config.h，此文件里面包含了相关寄存器配置的参数，和用户功能参数，如通道数选择，触摸门阈值，充电电压选择，基准电压选择等。用户可以修改此文件的参数，实现更好的触摸效果；

详细配置使用可查看文档“WS51F60/70 系列_触摸按键 PCB 应用设计”和“WS51F60/70 系列_触摸库开发环境和步骤介绍”。

4.20 其他

4.20.1 程序启动空间测试

通过寄存器 MECON 可设置软复位后程序启动空间，其中 BOOT=1，软复位后程序从外部 RAM 启动运行；BOOT=0，复位后程序从 Flash 启动运行。默认 BOOT=0 从 Flash 启动。代码如下，

```
/* 从外部 RAM 启动运行设置 */  
MECON |= 0x01; //boot, 外部 ram  
PCON |= 0x20;  
/* 从 Flash 启动运行设置 */  
MECON &= 0xfe; //boot, flash  
PCON |= 0x20;
```

5 芯片应用设计注意

由于芯片本身可能存在点设计问题，使用时应注意避免这些问题。

WS51F60/70 系列存在问题描述:

- (1) PWM 存在问题: PWM 独立输出模式下,使用 PWM1(/3/5)需要的打开 PWM0 和 PWM1 两个的使能,只开 PWM1/PWM0 的使能用不了;因此,使用时需要注意使用 PWM1 独立输出时需要同时打开 PWM0 和 PWM1 的运行使能。
- (2) PWM 输出时钟源时,运行使能存在问题:描述单开任意路使能,两路都有时钟源信号输出。
- (3) DAC 存在问题:选择输出内部 1.5V 基准电压时芯片无法输出。
- (4) IIC 存在问题: IIC 主从模式都不支持复合模式(restart)。